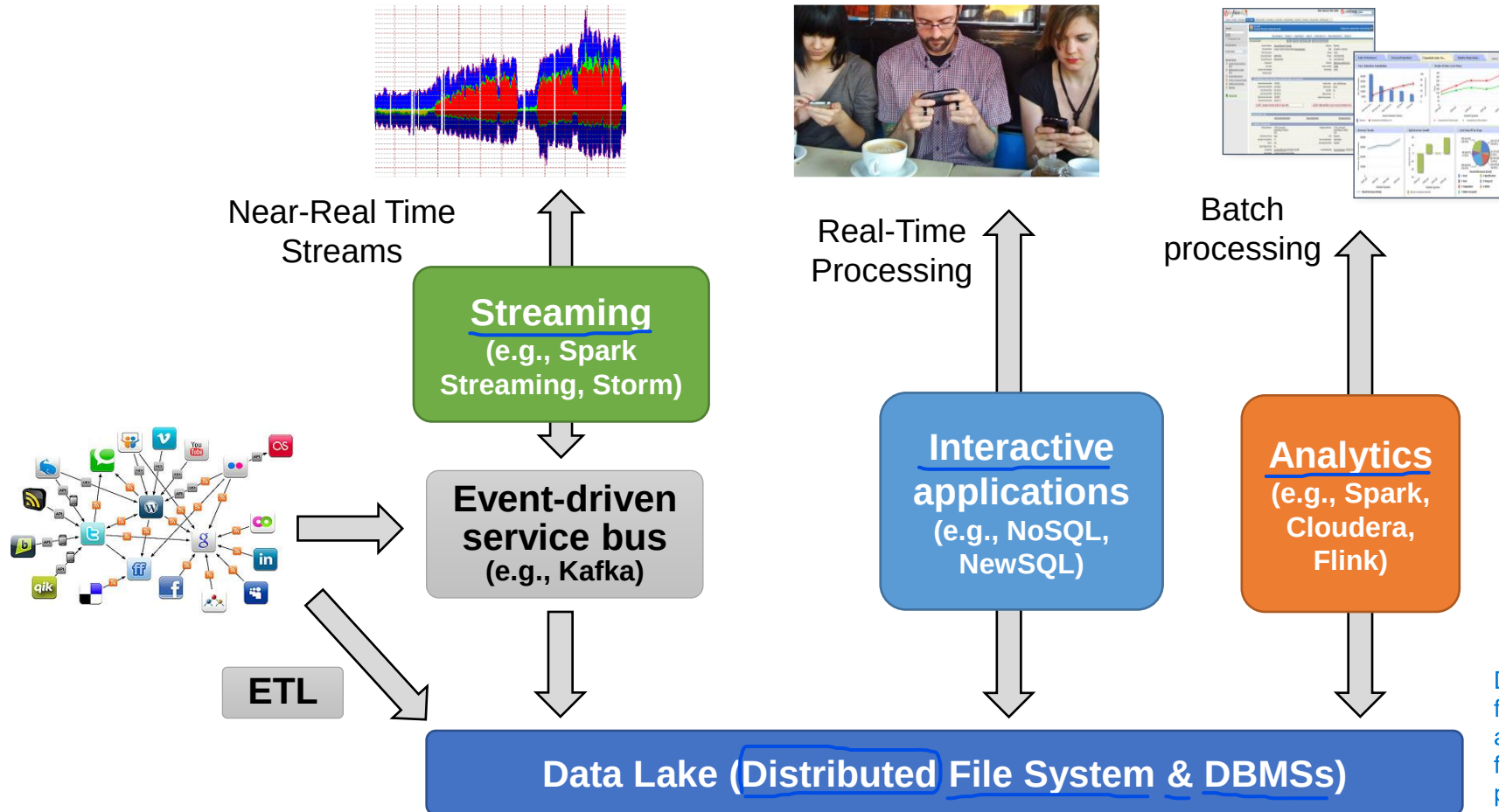


STORAGE:

Distributed File System

(H)DFS, File formats,
Data lake and lakehouse

The Big Data flow



Distributed File System is fundamental for Interactive and Batch processing, is not fundamental for Streaming processing (but in most cases you will save the data even in Streaming processing)

(H)DFS

Quando sappiamo come funziona il DFS di Hadoop (open-source), sappiamo bene o male gestire anche quelli proprietari, che sono delle estensioni di quello open-source

(Hadoop) Distributed File System

Goals of a DFS

Big data support

- Requirement: files may be too big to be processed by a single machine (or even bigger than single disks)

Abstraction

- Requirement: users should interact with disks on different machines as if they were one, unified disk

Fault-tolerance

- Requirement: disks can fail, machines can be unreachable, but data should always be available

"i problemi sono trasparenti all'utente": l'intero processo di gestione e mascheramento dei guasti avviene completamente dietro le quinte, senza che l'utente o l'applicazione client debbano preoccuparsene o subire interruzioni

HDFS

L'espressione 'streaming data access patterns' non ha nulla a che fare con "streaming analysis". Significa semplicemente che la modalità tipica di interazione con i file in HDFS consiste nel leggere sequenzialmente ed in modo continuo ampie porzioni di dati (se non l'intero file). L'architettura di HDFS è progettata per massimizzare il throughput (volume di dati trasferiti nell'unità di tempo) a discapito della latenza (tempo di risposta per accedere a un singolo dato casuale).

Nel mondo Big Data il termine Streaming viene usato con due significati molto diversi:

- Streaming Analytics / Real-Time Processing: Si riferisce all'elaborazione continua di flussi improvvisi di eventi in tempo reale (es. eventi Kafka, Spark Streaming, Flink).
- Streaming Data Access (in HDFS): Si riferisce alla modalità meccanica con cui un'applicazione legge un file dal disco system. Significa che il file viene aperto e "srotolato" (letto in modo sequenziale e ininterrotto come un nastro/flusso) dall'inizio alla fine o per ampi blocchi continui.

- Throughput (Capacità di trasmissione): Misura quanti dati il sistema riesce a leggere o scrivere nell'unità di tempo (es. Gigabyte al secondo). HDFS è ottimizzato al massimo per questo scopo: muovere grandi volumi di dati verso i motori di calcolo nel minor tempo complessivo possibile.

- Latenza (Tempo di risposta): Misura quanto tempo impiega il sistema per rispondere a una singola richiesta specifica (es. recuperare una riga di dati entro pochi millisecondi). HDFS non è adatto a query interattive ad altissima velocità o ad accessi punto-a-punto a bassa latenza; per quel tipo di workload si utilizzano invece i database NoSQL o i database relazionali tradizionali.

HDFS is a filesystem designed for storing **very large files** with **streaming data access** patterns, running on clusters of **commodity hardware**

- A typical file in HDFS is **gigabytes to terabytes in size**. There are Hadoop clusters running today that store petabytes of data
- **Applications that run on HDFS need streaming access to their data sets.** HDFS is designed more for batch processing rather than interactive use. The emphasis is on high throughput of data access rather than low latency of data access
- **HDFS applications need a write-once-read-many access model for files.** A file once created, written, and closed need not be changed. This assumption simplifies data coherency issues and enables high throughput data access
- **Hardware failure is the norm rather than the exception.** Therefore, detection of faults and quick, automatic recovery from them is a core architectural goal of HDFS

HDFS non è progettato per l'aggiornamento o la modifica diretta dei dati (in-place updates). Il suo funzionamento si basa sul modello Write-Once, Read-Many (WORM): un file viene creato, scritto e chiuso, senza la possibilità di cambiarne i singoli blocchi o record già salvati. L'unica operazione di scrittura successiva consentita è l'aggiunta di nuovi dati in coda (append); qualsiasi modifica a dati già esistenti richiede la riscrittura completa dell'intero file. HDFS si focalizza sull'archiviazione distribuita a basso costo e sull'accesso sequenziale ad alto throughput, delegando la gestione delle transazioni e delle modifiche puntuali ai DBMS.

HDFS: blocks

Why blocks?

- Files can be larger than disks.
- By splitting files into blocks, storage management is simplified and replication is easier

The block is the minimum amount of data read/written

(Blocks are small enough to be easily handled by a machine and large enough to contain a good quantity of data that can be sent with a high throughput)

- Disk blocks are normally 512B
- Filesystem blocks are typically a few KB
- HDFS blocks range between 64MB and 1GB (default: 128MB)
 - If a file is smaller, it will occupy the necessary disk space, not the full block size

→ Sebbene la dimensione logica del blocco sia grande (es. 128 MB), un file di dimensioni inferiori (es. 10 MB) occuperà su disco solo lo spazio effettivo necessario (10 MB) e non l'intera dimensione del blocco.

Why this big?

- Large files split into many small blocks require a huge number of seeks
- E.g.: block size = 4KB; file = 1GB; number of seeks = 250.000

(this will not be very optimised: troppo tempo per inviare il file)

→ tempi di ricerca del disco

Why not bigger?

HDFS: namenodes and datanodes

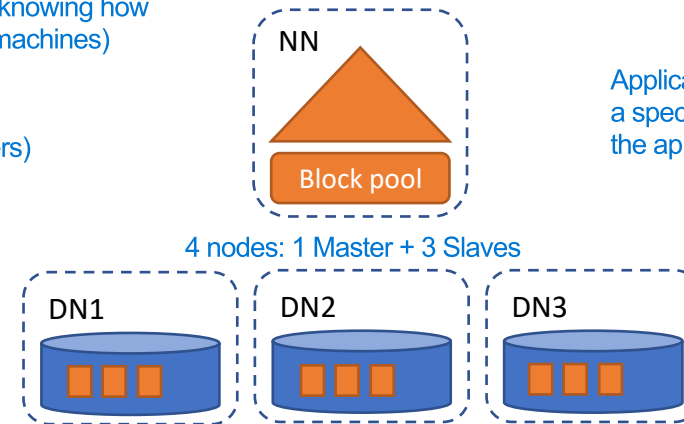
Nodes in an HDFS cluster operate in a master-slave pattern

Namenode (NN) - the master (the Service that is in charge of knowing how the data is distributed on other machines)

- Persistently maintains the filesystem tree and all files' and directories' metadata (organisation of folders)
- Keeps **in memory** the location of each block for a given file (*block pool*) (for each file, how many block and where are stored)

Datanodes (DNs) - the slave

- the Service that Store and retrieve blocks
- Periodically report to the NN with the list of blocks they are storing; *heartbeats* are sent to the NN to signal their active state (e.g., every 10 minutes)



HDFS: the SPoF

Il NameNode (NN) rappresenta il Single Point of Failure nell'architettura HDFS: gestendo l'albero del file system e la mappatura dei blocchi sui DataNode, la sua caduta rende l'intero cluster inaccessibile, anche se i dati fisici sui DataNode rimangono intatti.

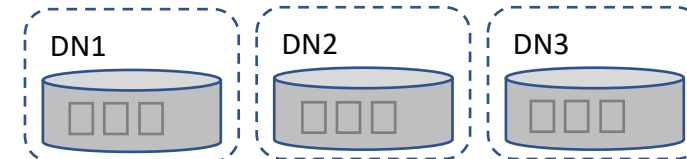
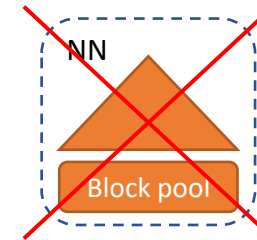
The NN is a single point of failure: without it, the filesystem cannot be used (no way to reconstruct the files from the blocks in the DNs)

1 **Backup solution** backup of the machine that runs the namenode (most basic solution, not the best one: it takes a lot of time to restart the NameNode)

- NN writes its persistent state to multiple filesystems, preventing loss of data

2 **Secondary NN solution** Il Secondary NameNode non è un NameNode di riserva pronto a subentrare in automatico.

- A separate machine regularly connects with the primary NN to build snapshots of its persistent data and saves them to local or remote directories
- These *checkpoints* can be used to restart a failed NN without having to replay the entire journal of file-system actions
- Still, restarting a NN could take 30+ minutes →



Sebbene il Secondary NameNode prevenga la perdita di dati, non offre disponibilità continua (High Availability): In caso di crash del NameNode primario, il passaggio o il riavvio non è istantaneo. La ricostruzione dello stato e il caricamento in memoria della mappa dei blocchi tramite l'ultimo checkpoint richiedono un intervento manuale o uno scatto di procedura che può impiegare 30 minuti o più, durante i quali il cluster rimane totalmente inattivo.

3 HDFS: High-Availability

High-Availability (HA) indicates a system that can tolerate faults

- The service never stops while the fault (be it hardware or software) is detected, reported, masked, and repaired off-line (errors are transparent to the users: system is quite-always available even if of failures)
- Most big data tools offer HA (not just HDFS)

Similar to the Secondary NN solution

HA is supported by configuring two separate machines as NNs

- One is in an *active* state, the other is in a *standby* state (the second one is continuously active (always synced with the first NN)). You can have 2+ NNs to have more redundancy and fault-tolerance
- In the event of a failure of the active NN, the standby NN takes over

The NN in standby keeps its metadata^{and the file system tree} up-to-date by:

- Reading the *edit log* files written by the active NN in a shared storage
 - Edit logs are typically stored in *Journal Nodes*, which are also replicated
- Receiving the data block locations^{and the file system tree} and the heartbeats directly from the DNs, which are configured to report to both NNs

(prende posto del primo NN, se cade, in modo immediato)

(more services up)

A HA solution is more efficient in recovering^(not so relevant), but requires more resources^(more services up) and communications^(not so relevant)

It isn't a Fault tolerant solution for NNs

HDFS: federation

Federation means that you have multiple Masters (NNs) that handle different portion of the File system (namespaces)

The size of the block pool^{and file system tree} is limited by the memory^(RAM) size of the NN

- May incur in scaling issues on large clusters with many files

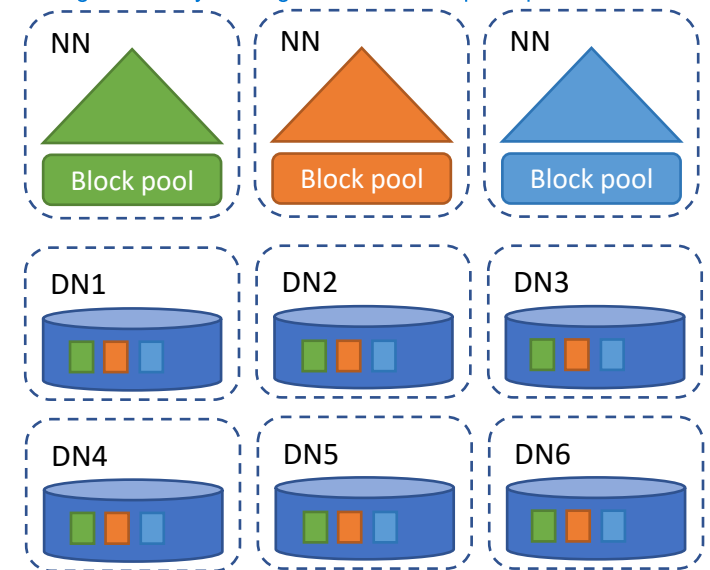
Solution: configure additional NNs

- Each NN manages a portion of the filesystem (*namespace*)
- NNs are independent of each other

Many advantages

- Performance** The Masters (NNs) are the ones that handle the communication with clients that ask for data access, so having multiple Masters can help handle multiple requests (helps even the division of the file system in namespaces)
 - NN can become a bottleneck for data access
- Availability**
 - In case of one NN's failure, the others are available
- Scalability**
 - By storing metadata in memory, the size of the NN's heap limits the number of files and blocks
- Maintainability, Security & Flexibility**
 - Each namespace is isolated and not aware of the others

Oltre alla federation, si può applicare, insieme ad esso, la soluzione High availability dove ogni NN ha la sua replica up-to-date



Cosa bisogna fare se i DataNodes falliscono? Quindi, bisogna rendere i dati disponibili (dati devono rimanere disponibili anche se il nodo che li gestisce va giù). La strategia tipica è quella di usare delle repliche dei blocchi

1 HDFS: replication

Each data block is independently replicated at multiple DNs in order to improve performance and robustness

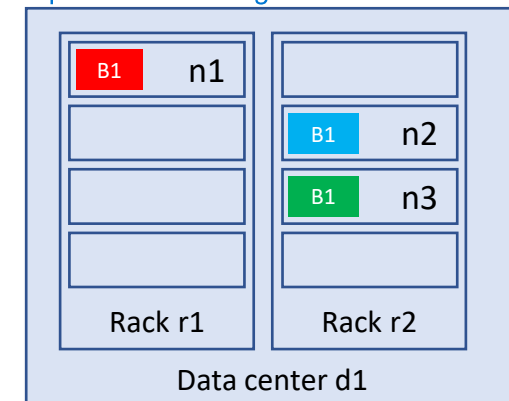
- Replication is aware of the cluster topology
- For each data block, the NN stores the list of DNs storing it

- Obiettivi della replicazione: Ogni blocco di dati viene salvato in copie multiple su DataNode distinti per garantire la sopravvivenza del dato in caso di rottura dei dischi o delle macchine, e per aumentare il parallelismo e le prestazioni delle letture distribuite.
- Consapevolezza della topologia (Rack Awareness): Il NameNode conosce la collocazione fisica dei DataNode all'interno dei singoli rack. Ciò consente di distribuire le repliche in modo strategico per sopravvivere ai guasti riguardanti al rack (es. la rottura di uno switch di rack), oltre a quello del nodo singolo.

The default replication factor is 3:

- Replica 1** is stored on the node (n1) where the client issued the write command (if the client resides within the cluster)
- Replica 2** is stored on a node (n2) in a rack (r2) different from the one of n1 (off-rack)
- Replica 3** is stored on a node, different from n2, but that belongs to r2

Contro: Significativo aumento dello spazio di archiviazione dovuto alla ridondanza dei dati.



Replicas can be rebalanced when nodes are added or unavailable

- Failover dei blocchi: Se un DataNode o un disco fallisce, il NameNode rileva la mancata ricezione degli heartbeat, nota che il fattore di replicazione è sceso sotto quota 3 e pianifica la copia del blocco mancante su un DataNode sano.

- Rebalancing del cluster: Quando vengono aggiunti nuovi DataNode vuoti, le repliche possono essere riequilibrate tra i nodi per evitare sbilanciamenti di memoria fisica o di I/O.

Un'altra strategia per il problema riguardante i failure dei DataNodes

2A HDFS: replication

L'Erasure Coding (EC) è una tecnica di Fault tolerance che sostituisce la duplicazione fisica dei blocchi con un calcolo matematico di parità.

HDFS supports Erasure Coding (EC) as an alternative to simple replication

- A mechanism similar to Redundant Array of Inexpensive Disks (RAID) 5-6
- Instead of replicating each block, blocks are striped
- A stripe is a sequence of chunks, composed of m data chunks + k parity chunks
- The data in a block may end up in multiple stripes

mechanism that you have on disks

i blocchi sono sottoposti a striping (i blocchi sono composti da più stripe)

di parità

possono essere distribuiti

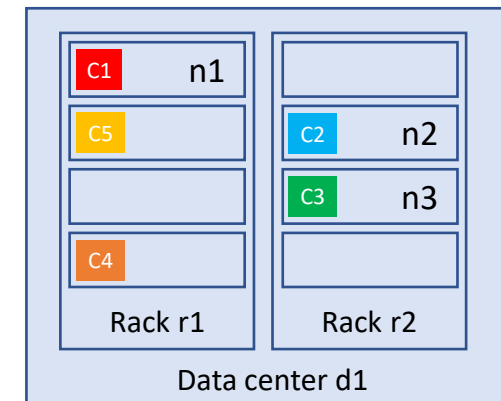
La quantità k viene decisa dagli amministratori di sistema in base al livello di sicurezza desiderato: Numero massimo di guasti tollerabili contemporaneamente = k
- Tolleranza ai guasti: Se si imposta $k = 3$, il sistema può tollerare la perdita o l'inaccessibilità simultanea di fino a 3 DataNode/dischi all'interno della stessa striscia senza perdere alcun dato.

I data chunk sono frammenti sequenziali costitutivi del dato originale. A ogni sequenza di m data chunk viene associato un set di k parity chunk: si tratta di blocchi di ridondanza calcolati algebricamente che garantiscono la tolleranza ai guasti, consentendo al sistema di ricostruire esattamente qualsiasi data chunk andato perso o danneggiato senza dover duplicare l'intero file.

Stripe

Chunk C1	1	0	0	1	1	1	0	1	0
Chunk C2	0	0	0	1	1	1	0	1	1
Chunk C3	1	0	1	0	1	0	1	0	0
Chunk C4	1	1	0	0	0	0	1	1	1
Parity C5	1	1	1	0	1	0	0	1	0

In questo caso, abbiamo 4 data chunks e 1 parity chunks ed i 5 DataNode salvano ognuno un data chunk o parity chunk



I chunk di uno Stripe sono distribuiti su rack differenti (es. C1 nel DataNode n1, C4, C5 nel Rack r1; C2, C3 nei DataNode n2 e n3 del Rack r2) per far sì che il sistema sopravviva anche al guasto di un intero rack.

Lo Stripe è l'unità logica orizzontale composta dall'insieme di tutti i m data chunk + i k parity chunk: Nell'immagine, lo Stripe è l'intera riga orizzontale che comprende C1+C2+C3+C4+C5. I dati di un data block possono estendersi su più Stripe successivi.

<https://docs.cloudera.com/runtime/7.3.1/scaling-namespaces/topics/hdfs-ec-overview.html>

C5 è il risultato di un'operazione logica (es. XOR / Reed-Solomon) applicata colonna per colonna sui bit di C1, C2, C3, C4. Se C2 va perso a causa del guasto di n2, il sistema combina i bit superstiti di C1, C3, C4 e C5 per ricalcolare e ripristinare esattamente la sequenza di C2.

In this case, 1 if odd, 0 if even

← additional data

Non contiene dati dell'utente, ma un calcolo matematico effettuato sui dati di C1, C2, C3, C4

2_B HDFS: replication

Erasure Coding (EC) works best for cold datasets with relatively low I/O activities

EC è progettato principalmente per i cosiddetti cold datasets, ovvero volumi di dati storici o di archivio poco attivi, caratterizzati da basse frequenze di lettura e scrittura (basso I/O). Poiché EC scambia lo spazio su disco con un maggiore consumo di CPU e banda di rete, applicarlo su hot datasets (molto frequentati) penalizzerebbe le prestazioni del cluster.

Main advantages

Main reason for EC

- Sensibly reduce data redundancy (from 200% to 50% with default setups)

Rispetto alla prima soluzione, EC richiede meno spazio (viene replicata solo una parte del dato totale)

- If m parity chunks are created every k chunks, the redundancy factor is m/k

- The default is 3 parity chunks every 6 chunks

La configurazione Default di EC è 6 data chunk e 3 parity chunk per Stripe

- Faster writes (chunks are distributed)

Poiché i vari chunk che compongono la striscia vengono distribuiti ed emessi in parallelo verso molteplici DataNode fisici contemporaneamente, la velocità di scrittura sfrutta la larghezza di banda e l'I/O combinato di più dischi in parallelo.

Main disadvantages

- Higher CPU cost in both reading and writing

In Scrittura, il sistema deve eseguire algoritmi algebrici (Reed-Solomon) su matrici di bit per calcolare i chunk di parità prima dell'invio. In Lettura, la ricomposizione delle strisce richiede overhead computazionale da parte delle CPU dei nodi.

- Longer recovery time in case of failure

Con EC, il ripristino di un chunk perso richiede di scaricare via rete i restanti chunk dello stripe da tutti gli altri nodi e ri-calcolare algebricamente il dato mancante tramite CPU.

- Loss of data locality (to read a block, you need to get its stripes from multiple machines)

A differenza della prima soluzione di replicazione, con EC accedere ad un blocco richiede di recuperare i singoli chunk distribuiti su più macchine differenti per poter ricomporre il dato originale. Questo approccio comporta un overhead architetturale maggiore rispetto alla replicazione semplice: non è più possibile leggere un intero blocco in locale dallo stesso nodo di calcolo, costringendo il sistema ad accedere sempre alla rete. La ricomposizione dei chunk e l'eventuale decodifica algebrica dei dati richiedono più risorse computazionali e generano un traffico di I/O diffuso tra i nodi del cluster

Con EC, poiché un blocco è frammentato in strisce su macchine diverse, il sistema è costretto ad accedere sempre alla rete per raccogliere i singoli chunk e ricomporre il blocco logico.

Goals of a DFS

Big data support

- Requirement: files may be too big to be processed by a single machine (or even bigger than single disks)
- Typical solution: split files into smaller blocks, store blocks on different machines

Abstraction

- Requirement: users should interact with disks on different machines as if they were one, unified disk
- Typical solution: master-slave architecture with high-availability Client communicates only with the Master and the whole system handles the distribution, retrieval of the data

Fault-tolerance

- Requirement: disks can fail, machines can be unreachable, but data should always be available
- Typical solution: store multiple copies of each block (replication)

HDFS: not always the best fit

Although this may change in the future, there are areas where HDFS is not a good fit today

Low-latency data access

Accesso ai dati a bassa latenza

- HDFS is optimized for delivering a high throughput of data, and this may be at the expense of latency. Applications that require access to data in the tens of milliseconds range will not work well with HDFS.

andare a discapito della

Lots of small files

Grande quantità di file piccoli

- The limit to the number of files in a filesystem is governed by the amount of memory on the NN, because it holds filesystem metadata in memory.
- ~~Although~~ storing millions of files is feasible, billions is beyond the capability of current hardware.

determinato dalla

RAM

Sebbene

HDFS: I/O

An application client wishing to read a file (or a portion thereof) must first contact the NN to determine where the actual data is stored

In response to the client request the NN returns:

- The relevant block id
- The location where the block is held (i.e., which DN)

The client then contacts the DN to retrieve the data

Blocks are themselves stored on standard single-machine file systems

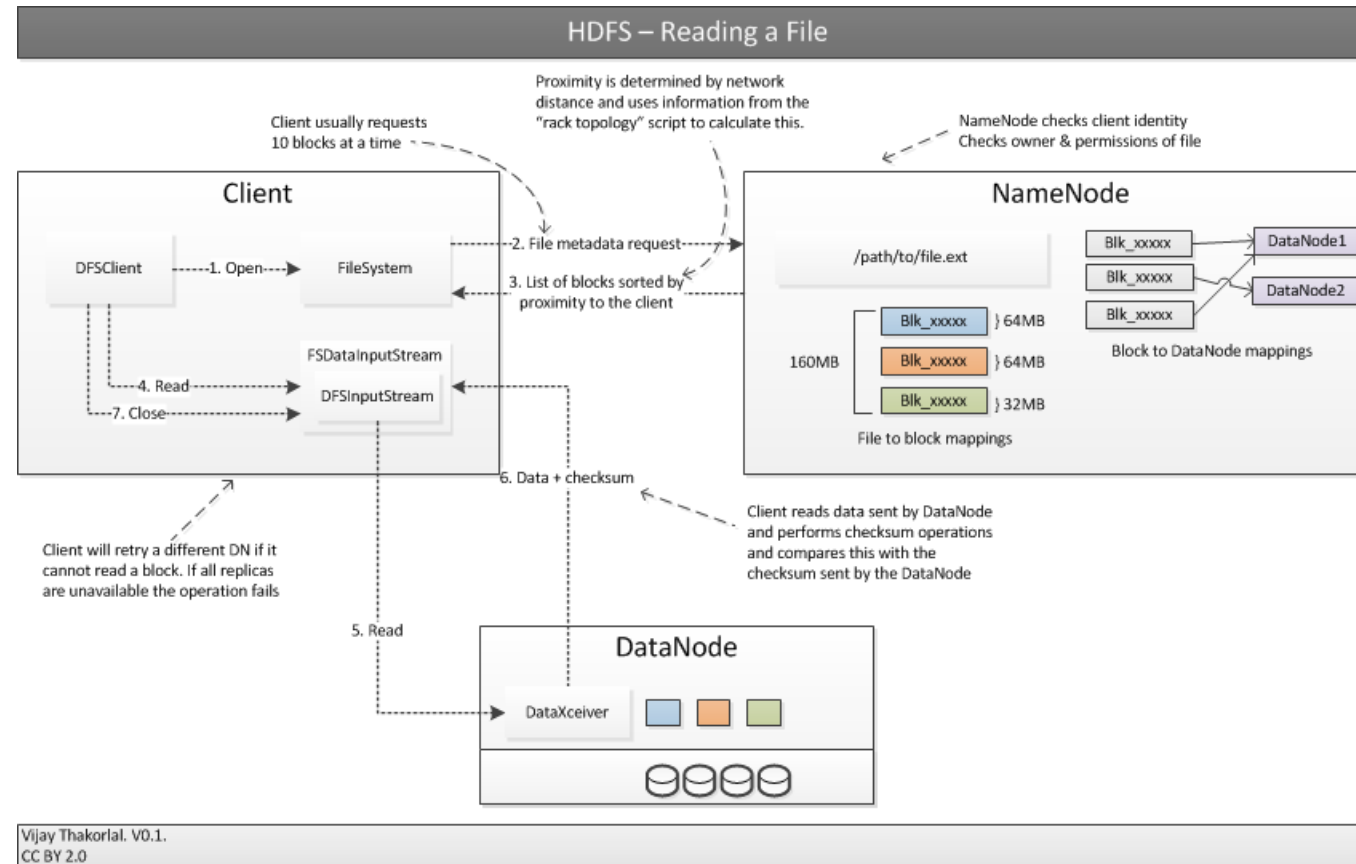
- HDFS lies on top of the standard OS stack

Important features of the design:

- Data is never moved through the NN
- All data transfer occurs directly between clients and DNs
- Communications with the NN only involve transfer of metadata

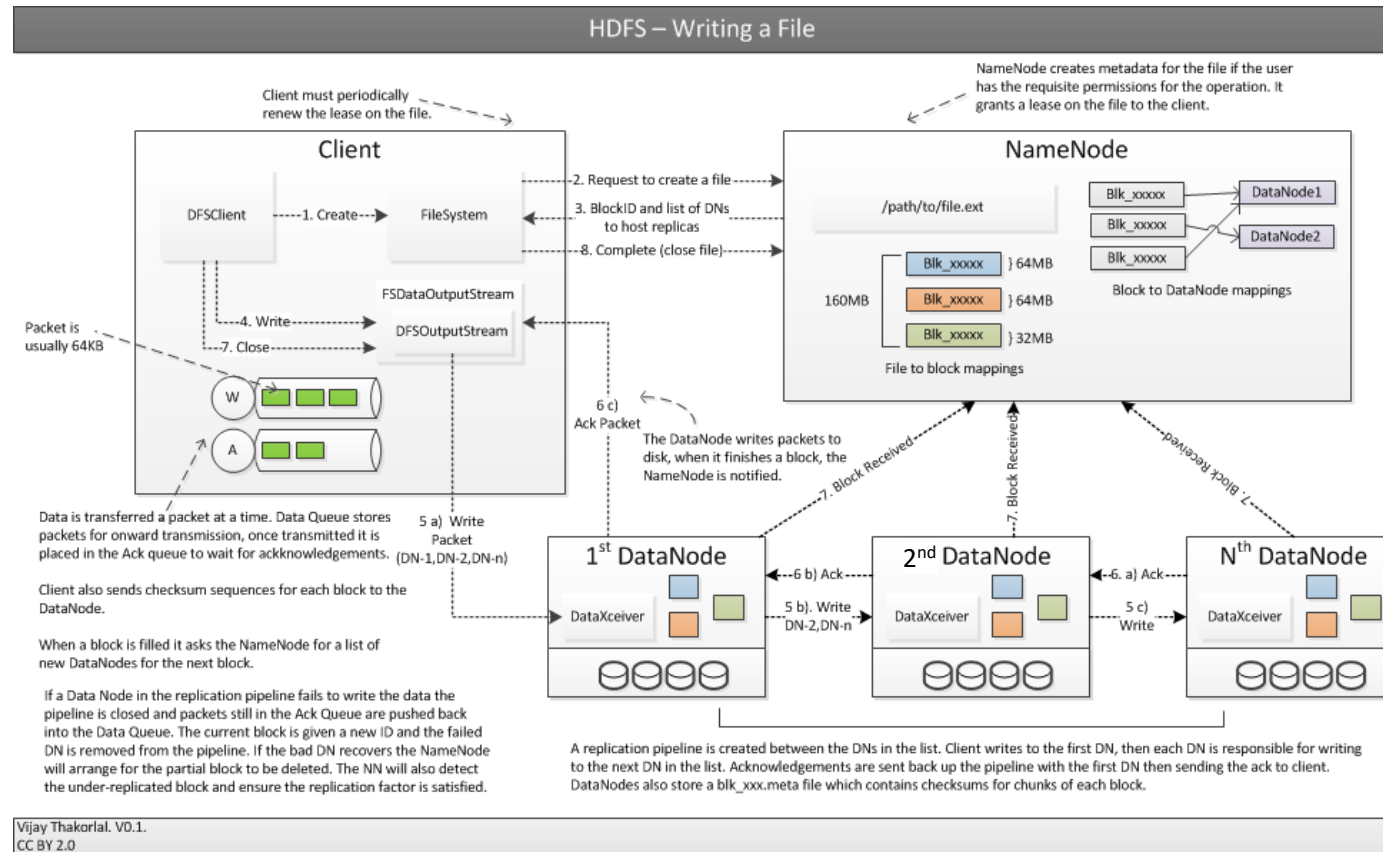
Reading in HDFS

Only to see the sequence of communication



Writing in HDFS

Only to see the sequence of communication



File formats

Standard formats are not suitable

Row-oriented vs Column-oriented

Apache Parquet for nested structures

File formats

Data usually comes in standard file formats

- Structured CSV files
- Semi-structured JSON files
- Unstructured textual files

Sebbene questi formati siano molto diffusi per la loro semplicità e leggibilità umana, non sono adatti alle infrastrutture Big Data.

However, there are file formats more suitable for big data infrastructures and applications

Per elaborare petabyte di dati in modo efficiente, le architetture Big Data richiedono formati specializzati che siano:

- Splittable file formats
- Column-oriented (vs row-oriented) file formats

- Frazionabili (Splittable): In grado di essere divisi e letti in parallelo da più nodi di calcolo contemporaneamente.
- Orientati alle colonne (Column-oriented): Capaci di organizzare i dati per colonna anziché per riga.

Questa slide illustra le tre motivazioni fondamentali per cui i formati di file generici (come CSV o JSON) risultano inefficienti in ambito Big Data, e come i formati nativi per Big Data (come Parquet) risolvano questi problemi.

Why using big data formats

Nei formati di testo tradizionali (JSON, CSV), ogni singola cifra, carattere o spazio occupa un byte (o più) di memoria ASCII/UTF-8:

- Formato Testo (JSON/CSV): Il numero 12345678 salvato come testo occupa 8 byte (un byte per ciascuna cifra testuale). Se poi è racchiuso tra virgolette o dentro un oggetto JSON come {"deposit_money": "12345678"}, si aggiungono ulteriori byte inutili per le chiavi, i due punti e le parentesi.
- Formato Binario: In rappresentazione binaria nativa (intero a 32 bit), lo stesso numero 12345678 occupa sempre e solo 4 byte, indipendentemente da quante cifre ha.

Quando un motore di calcolo legge un file CSV o JSON, deve convertire continuamente le stringhe di testo nel tipo di dato nativo in memoria (es. da stringa "12345678" a intero a 32 bit), generando un pesante carico sulla CPU. Nei formati binari i dati sono già codificati nel loro tipo nativo, eliminando del tutto l'overhead di parsing.

1 Binary serialization

- Compact streams of bytes occupy less space
- Without it, you incur in large overhead for storing data as text
 - More space required
 - Type-conversion required when reading and writing

{ "deposit_money": "12345678" }	
JSON	Binary
'0x6d', '0x6f', '0x6e', '0x65', '0x79', '0x31', '0x32', '0x33', '0x34', '0x35', '0x36', '0x37', '0x38'	'0x01', '0xBC614E'

2 Splittability

La frazionabilità è la capacità di dividere un singolo file in porzioni autonome e indipendenti che possono essere lette in parallelo da nodi diversi di un cluster.

- Metadata headers allow to skip unnecessary I/O
- Without it, you cannot access single portions of files

I formati per Big Data integrano header e footer contenenti metadati e indici di posizione (i metadati vanno nel file o blocchi del file). Questa struttura consente ai motori di calcolo (es. Apache Spark o MapReduce) di applicare l'I/O skipping, accedendo direttamente ai soli byte d'interesse ed evitando la scansione dei dati irrilevanti

3 Compression

Nei file tradizionali (es. un CSV compresso in .zip o .tar.gz), l'algoritmo impacchetta il file come un unico blocco monolitico (e poi, se usa i blocchi, split il file compresso in blocchi: quindi, ogni volta devi riprendere tutti i blocchi e decomprimere). Se una query richiede un solo record, l'intero archivio deve essere decompressa per poterlo estrarre. I formati Big Data introducono invece la compressione granulare:

- Block-level: distinctly compress the single blocks
- Record-level: distinctly compress the single records within a block
- Otherwise, the whole file must be compressed and decompressed for reading

Ogni blocco del file viene compresso in modo indipendente dagli altri. Il sistema decomprime soltanto il blocco specifico contenente le righe richieste dalla query.

La compressione viene applicata su singoli record o colonne all'interno di un blocco, ottimizzando ulteriormente lo spazio.

Row-oriented vs Column-oriented

Disks store data sequentially (they are 1-dimensional) →

- Tables cannot be directly represented
- Cells are stored either row-by-row or column-by-column

I supporti di memoria (HDD e SSD) memorizzano i byte in modo sequenziale su una linea unidimensionale. Una tabella dati, essendo bidimensionale (composta da righe e colonne), non può essere salvata così com'è su disco, ma deve essere "appiattita" (linearizzata). La scelta di come linearizzare i dati determina le prestazioni del sistema in base al tipo di carico di lavoro: Row-oriented o Column-oriented

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

Row-oriented

Le celle vengono memorizzate riga per riga ([A1, B1, C1], poi [A2, B2, C2], ecc.). Tutti gli attributi di un singolo record risiedono nello stesso blocco fisico contiguo.

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

Column-oriented

Le celle vengono memorizzate colonna per colonna ([A1, A2, A3], poi [B1, B2, B3], ecc.). Tutti i valori di una specifica colonna risiedono nello stesso blocco fisico contiguo.

Online
Transaction
Processing

Row orientation is better for transactional workloads (OLTP)

- Applications work on a row basis (e.g., updating a customers' profile, adding a new order)

Modalità di accesso: L'applicazione lavora per singola riga/entità (inserimento, modifica o lettura di un intero record). Perché conviene: Poiché tutti i dati del cliente o dell'ordine si trovano nello stesso blocco di disco, il sistema legge o scrive l'intera riga con una singola operazione d'I/O.

Column orientation is better for analytical workloads (OLAP)

- Analyses are done on a columnar basis (e.g., the average salary of employees by department)

Modalità di accesso: Le analisi lavorano per colonna ed eseguono aggregazioni su grandi volumi di dati. Perché conviene: Se una tabella contiene 100 colonne ma la query di analisi necessita solo di 2, il formato columnar legge dal disco soltanto i blocchi dedicati a quelle due colonne. Le altre 98 colonne vengono ignorate del tutto.

Online
Analytical
Processing

In Big Data, OLAP is the most done

Row-oriented vs Column-oriented

There are several advantages to column-oriented (columnar) file formats

- **Better compression**
 - **Data is more homogenous** Poiché le celle vengono memorizzate colonna per colonna anziché riga per riga, tutti i dati contenuti nello stesso blocco fisico appartengono allo stesso tipo e presentano spesso valori simili o ripetuti. Gli algoritmi di compressione funzionano con un'efficienza nettamente superiore sui dati omogenei.
 - The space savings are very noticeable at the scale of a Hadoop cluster
 - Type-specific encodings è possibile applicare tecniche di codifica studiate appositamente per la natura del tipo di dato
- **Reduced I/O for analytical queries**
 - **Only a subset of the columns are read**
 - Skip unnecessary deserialization
- **Operate on encoded data** (e.g., when a dictionary encoding is used)
 - ↳ I motori di calcolo possono lavorare direttamente sui valori codificati senza doverli decodificare preventivamente

Row-oriented file formats

Many formats

- **SequenceFiles** one of the first file formats of Hadoop (outdated)
 - Binary key-value pairs
 - Specifically designed to work with MapReduce (but others are available; e.g., MapFiles)
 - No support besides Java
 - Mostly used for Intermediate data storage and to packing many small files into a single SequenceFile
- **Apache Thrift** (Facebook), **Protocol Buffers** (Google)
 - Born to facilitate communication between programs (RPC) then used in Big Data scenarios
 - **Multi-language support** Permettono a programmi scritti in linguaggi diversi di scambiarsi strutture dati in formato binario.
 - Compilazione dello Schema: Richiedono la definizione di uno schema formale tramite file di configurazione con sintassi specifica. Tali file devono essere compilati con strumenti esterni per generare il codice sorgente (classi) nel linguaggio target.
 - Limiti nei Big Data: Sebbene molto veloci e compatti, la necessità di ricompilare il codice a ogni variazione della struttura dati rende la gestione delle pipeline di analisi più rigida.
 - Key features are available only through external libraries
- **Apache Avro**
 - **Addresses the major drawbacks of Writable** (the default serialization of **SequenceFiles**)
 - **Multi-language support**
 - **More integrated with Hadoop**
 - **Supports schema evolution, no need to compile code**
 - Nessuna Compilazione Necessaria: A differenza di Thrift e Protobuf, non occorre compilare preventivamente il codice per accedere ai dati o leggere il file. I programmi possono leggere lo schema direttamente dall'header ed interpretare i dati dinamicamente.
 - Supporto alla Schema Evolution: Se lo schema cambia nel tempo (es. viene aggiunto o rinominato un campo), Avro gestisce automaticamente la riconciliazione tra lo schema usato per la scrittura e quello usato per la lettura.

Row-oriented file formats

In a separate file, you define what is the structure of the dataset that you want to store

Rispetto a Thrift e Protobuf, Avro integra lo schema di formattazione direttamente nei metadati del file stesso. Questo elimina la necessità di compilare classi specifiche per ogni linguaggio di programmazione prima dell'accesso ai dati.

- Svantaggio: L'overhead dovuto alla lettura dello schema rende le operazioni di I/O leggermente più lente rispetto alla velocità di Thrift o Protobuf.
- Vantaggio: Semplifica notevolmente la schema evolution. Avro è in grado di gestire autonomamente file scritti con schemi diversi nel tempo, risolvendo automaticamente campi mancanti o aggiuntivi durante la deserializzazione.

In Avro, data is always serialized with its schema

- Each file is self-describing
- Different schemas can be used for serialization and deserialization (Avro will autonomously handle the missing / extra / modified fields)

Thrift

```
namespace java serializers.thrift.media

typedef i32 int
typedef i64 long

enum Size {
    SMALL = 0,
    LARGE = 1,
}

enum Player {
    JAVA = 0,
    FLASH = 1,
}

struct Image {
    1: string uri, //url to the images
    2: optional string title,
    3: required int width,
    4: required int height,
    5: required Size size,
}

struct Media {
    1: string uri, //url to the thumbnail
    2: optional string title,
    3: required int width,
    4: required int height,
    5: required list<string> person,
    6: required Player player,
    7: optional string copyright,
}

struct MediaContent {
    1: required list<Image> image,
    2: required Media media,
}
```

Protobuf

```
package serializers.protobuf.media;

option java_package = "serializers.protobuf.media";
option java_outer_classname = "MediaContentHolder";
option optimize_for = SPEED; affects the C++ and Java
code generators

message Image {
    required string uri = 1; //url to the thumbnail
    optional string title = 2; //used in the html
    required int32 width = 3; // of the image
    required int32 height = 4; // of the image
    enum Size {
        SMALL = 0;
        LARGE = 1;
    }
    required Size size = 5;
}

message Media {
    required string uri = 1;
    optional string title = 2;
    required int32 width = 3;
    required int32 height = 4;
    repeated string person = 5;
    enum Player {
        JAVA = 0;
        FLASH = 1;
    }
    required Player player = 6;
    optional string copyright = 7;
}

message MediaContent {
    repeated Image image = 1;
    required Media media = 2;
}
```

The separate file is then
✓ **compile**

protoc -I=\$SRC_DIR
--java_out=\$DST_DIR
\$SRC_DIR/addressbook.proto

The separate file has
schema information to
obtain classes
with getters
and setters

com/example/tutorial/
AddressBookProtos.java

and then you use these classes
in order to access the data that is
stored according to this schema
(if there is any change of the
schema, you need to modify the
separate class and recompile)

Column-oriented file formats

Many formats

- **ORC Files** (Optimized Record Columnar Files)
 - First columnar format on Hadoop (initially *RC Files*)
 - No support for schema evolution
 - Originally designed for Hive alone
- **Apache Parquet** the most used. More expressive than ORC Files
 - General-purpose
 - Supports schema evolution

Parquet

Store in a columnar way data
(tables, JSON: objects, arrays ...)
that has a nested structure

A columnar storage format for efficient querying of **nested structures** in a **flat format**

A little history:

- 2006: the *Dremel* system, using *ColumnIO* file format, goes in production at Google
- 2010: Google publishes a paper about Dremel
- 2013: Apache Parquet 1.0 is released (open source) as a joint effort between Twitter and Cloudera

(you can use it)

Parquet ensures interoperability through several frameworks (Spark, MapReduce, etc.) and query engines (Hive, Impala, Pig, Presto, etc.)

Parquet: data model

Root: a group of fields called a **message**

Field types: **group** or **primitive** (e.g., string)

Data type (string, booleans, numbers, ...)

A primitive store data. Group can contain other fields (can nest groups)

Frequency: **required** (exactly one), **optional** (zero or one), or **repeated** (zero or more)

```
message AddressBook {  
    required string owner;  
    repeated string ownerPhoneNumbers;  
    repeated group contacts {  
        required string name;  
        optional string phoneNumber;  
    }  
}
```

Elementi Fondamentali dello Schema

- Root (message) è l'elemento radice dell'intero schema, l'equivalente di una tabella o di un record principale. Ogni file Parquet definisce un message che contiene la gerarchia di tutti i campi.

- Tipi di Campo (Field Types):

> Primitive: Tipi di dato atomici di base (es. string, int32, boolean, double).

> Group: Strutture o contenitori logici usati per raggruppare altri campi (siano essi primitivi o altri group annidati).

- Frequenza / Cardinalità (Repetition Types):

Definiscono quante volte un campo può comparire all'interno della struttura:

> required: Il campo deve essere obbligatoriamente presente esattamente una volta. Non può mai assumere valore null.

> optional: Il campo può comparire zero o una volta. Può essere privo di valore (null).

> repeated: Il campo può comparire zero, una o più volte. È il meccanismo nativo usato da Parquet per rappresentare liste, array ed elenchi.

Each record is called a Message (a row). A message can contain multiple fields

Protocol Buffers and Thrift have a similar (but more complex) schema, which defines also Maps, Lists and Sets

Parquet: data model examples

<u>Schema: List of Strings</u>	<u>Data: ["a", "b", "c", ...]</u>
<pre>message ExampleList { <u>repeated</u> <u>string</u> list; }</pre>	<pre>{ list: "a", list: "b", list: "c", ... }</pre>

<u>Schema: Map of strings to strings</u>	<u>Data: ["AL" => "Alabama", ...]</u>
<pre>message ExampleMap { <u>repeated</u> <u>group</u> map { <u>required</u> <u>string</u> <u>key</u>; <u>optional</u> <u>string</u> <u>value</u>; } }</pre> In JSON file, you cannot have a null on the key You cannot enforce that the key must be only one in the map	<pre>{ map: { key: "AL", value: "Alabama", }, ... }</pre>

Parquet: unnesting

Appiattimento delle strutture annidate

Prima di calcolare i livelli per i singoli valori, bisogna determinare il Max DL e il Max RL per ciascuna colonna foglia basandosi sullo schema:

- Max Definition Level (Max DL) indica quanti campi optional o repeated si incontrano lungo il percorso dalla radice dello schema fino alla colonna foglia. I campi required non incrementano il Max DL, poiché non possono mai assumere valore null se il loro contenitore genitore esiste.
- Max Repetition Level (Max RL) indica quanti campi repeated, ovvero liste/array, si incontrano lungo il percorso dalla radice alla colonna foglia.

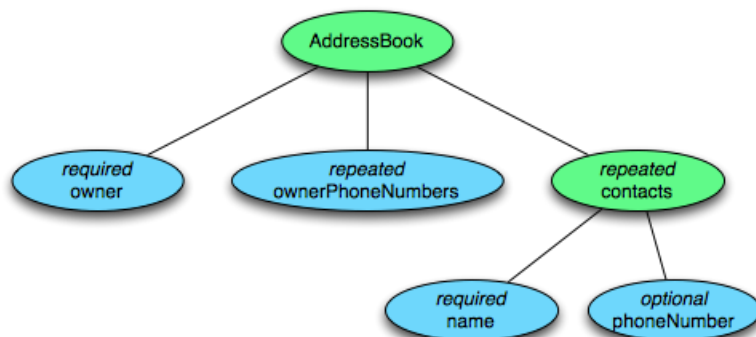
Problem: can we store nested data structures in columnar format?

- We need to map the schema to a list of columns in a way that we can write records to flat columns and read them back to their original nested data structure
- With repeatable fields, how do I know when one record ends and another begins?

One column per primitive type field in the schema

- If we represent the schema as a tree, the primitive types are the leaves of this tree).

Parquet rappresenta lo schema di un record annidato come un albero: I nodi interni dell'albero sono le strutture o i gruppi (group). Le foglie dell'albero sono unicamente i tipi primitivi (es. string, int). Per risparmiare spazio e ottimizzare l'I/O, Parquet crea una colonna fisica solo per le foglie dell'albero. I gruppi e le strutture intermedie non generano colonne fisiche su disco.



Column	Type
owner	string
ownerPhoneNumbers	string
contacts.name	string
contacts.phoneNumber	string

AddressBook			
owner	ownerPhoneNumbers	contacts	
		name	phoneNumber
...	...	...	...
...	...	...	...
...	...	...	...

Solution

- Each value is associated with two integers: **repetition level** and **definition level**
- These integers allow to fully reconstruct the nested structures while still being able to store each primitive separately

di ogni colonna

Parquet: definition level

Il DL misura quanti campi optional/repeated sono effettivamente definiti (diversi da null) per quel dato valore:

- Se il valore foglia è presente: Tutti i campi del percorso sono definiti, quindi DL = Max DL.
- Se il valore foglia è null: Il DL indica a che livello della gerarchia la catena si è interrotta:
 - > DL=0: L'oggetto radice o il primo gruppo opzionale/ripetuto è null.
 - > DL=1: Il primo livello esiste, ma il secondo livello è null, e così via.

The example data contains one column, "**a.b.c**", where all fields are optional and can be *null*.

When **c** is defined, then necessarily **a** and **b** are defined too, but when **c** is *null*, we need to save the level of the *null* value.

- There are 3 nested optional fields, so the maximum definition level is 3.

```
message ExampleDefLevel {  
  optional group a {  
    optional group b {  
      optional string c;  
    }  
  }  
}
```

At which level the null is defined?

Value	Definition Level
a: null	0
a: { b: null }	1
a: { b: { c: null } }	2
a: { b: { c: "foo" } }	3 (actually defined)

Spiegazione

- a è nullo (assenza al primo livello).
- a esiste, ma b è nullo.
- a e b esistono, ma c è nullo.
- Tutti e 3 i livelli sono definiti, il valore "foo" è presente.

Questi numeri dipendono da quanto è complessa la struttura dell'albero (in questo caso, il numero va da 0 a 3) e quali gruppi sono opzionali o no (i campi required non hanno bisogno del definition level).

Parquet: definition level

The maximum definition level is now 2, as **b** does not need one. The value of the definition level for the fields below b changes as follows:

```
message ExampleDefLevel {  
  1 optional group a {  
    required group b {  
      2 optional string c;  
    }  
  }  
}
```

Value	Definition Level
a: null	0
a: { b: null }	Impossible, as b is required
a: { b: { c: null } }	1
a: { b: { c: "foo" } }	2 (actually defined)

Definition level is only used
in presence of optional field

Remember: the smaller the definition levels, the less the required bits

Dichiarare un campo come required (obbligatorio) anziché optional evita di incrementare il Max DL, poiché quel campo non potrà mai essere nullo. Evitare annidamenti opzionali superflui mantiene basso il livello massimo di definizione.

Poiché Parquet applica la tecnica del Bit Packing per memorizzare questi interi, il numero di bit necessari per salvare ogni singolo DL dipende esclusivamente dal valore massimo che il DL può raggiungere (Max Definition Level).

Parquet: repetition level

L'RL risponde alla domanda: "In quale punto della gerarchia stiamo iniziando una nuova ripetizione?"

- RL=0 indica che stiamo iniziando un nuovo record radice (un nuovo oggetto principale).
- RL=1 indica che stiamo aggiungendo un nuovo elemento nella prima lista (repeated) incontrata nel percorso da radice a foglia.
- RL=2 indica che stiamo aggiungendo un nuovo elemento nella seconda lista (repeated) annidata, e così via.

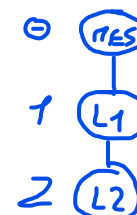
The presence of repeated fields requires to store *when* new lists are starting in a column of values. Repetition level indicates **at which level we have to create a new list for the current value.**

When I see different phone numbers, do they belong to a new record or to the record I'm already looking at?

Schema:	Data: [[a,b,c],[d,e,f,g]],[[h],[i,j]]
<pre>message nestedLists { repeated group level1 { repeated string level2; } }</pre>	<pre>{ level1: { level2: a level2: b level2: c }, level1: { level2: d level2: e level2: f level2: g } }, { level1: { level2: h }, level1: { level2: i level2: j } }</pre>

The repetition level can be seen as a marker of when to start a new list and at which level.

Repetition level	Value
0	a
2	b
2	c
1	d
2	e
2	f
2	g
0	h
1	i
2	j



0 marks every new record;

implies creating a new level1 and level2 list

1 marks every new level1 list;

implies creating a new level2 list as well

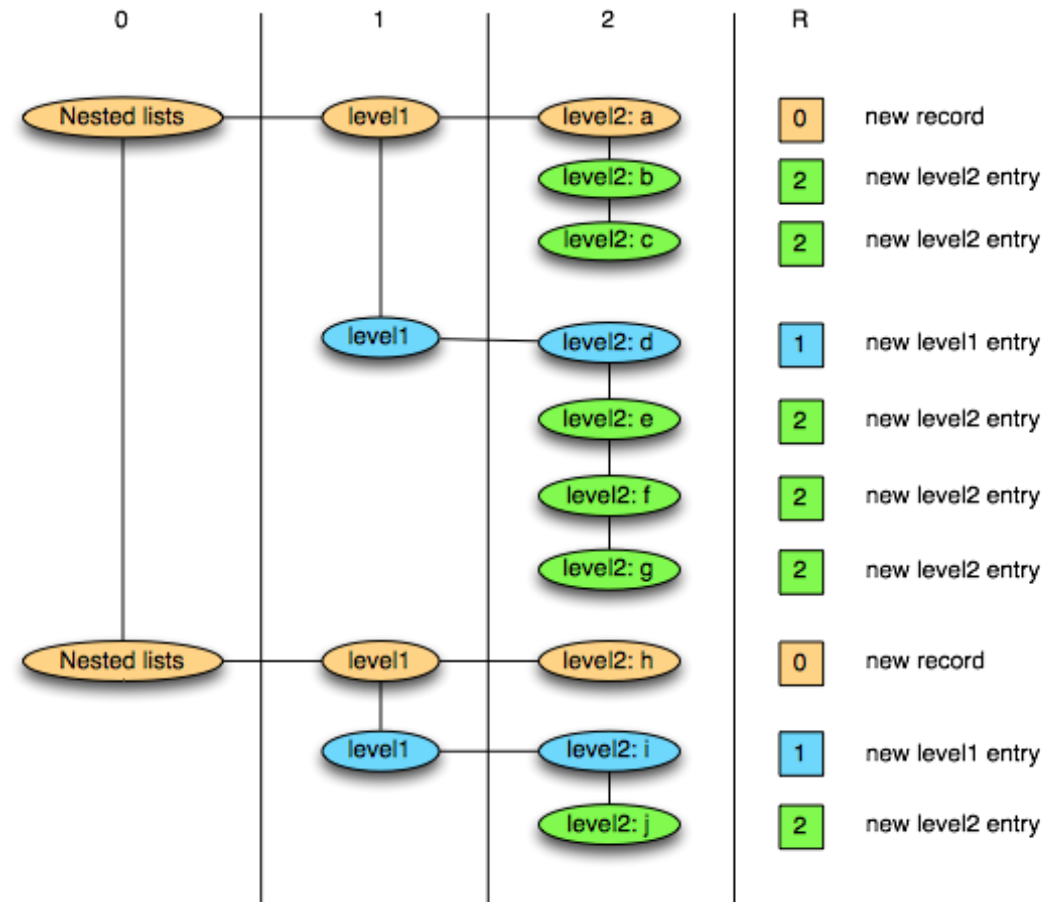
2 marks every new element in a level2 list

- RL=0: Marca l'inizio di un nuovo record radice (una nuova riga/oggetto principale).

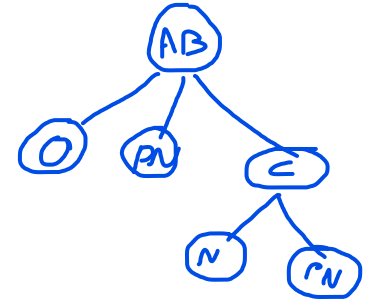
- RL=1: Marca l'inizio di un nuovo elemento nella prima lista (repeated di livello 1) dello stesso record.

- RL=2: Marca l'inizio di un nuovo elemento in una lista annidata interna (repeated di livello 2).

Parquet: repetition level



Parquet: AddressBook example



Definition level for column "contacts.phoneNumber"

- Defined phone number: 2
- A contact without phone number: 1
- No contacts: 0

ownerPhoneNumbers (repeated string):

- Max DL = 1: È presente 1 campo repeated lungo il percorso radice-foglia. Mantiene traccia se la lista contiene elementi (DL=1) o se è vuota (DL=0).
- Max RL = 1: È presente 1 solo livello di annidamento della lista (repeated). Un valore con RL=0 indica un nuovo proprietario (AddressBook), mentre RL=1 indica un ulteriore numero di telefono per lo stesso proprietario.

is required (so, no need for DL and RL)

Column	Max Definition level	Max Repetition level
owner	0 (owner is <i>required</i>)	0 (no repetition)
ownerPhoneNumbers	1	1 (<i>repeated</i>)
contacts.name	1 (name is <i>required</i>)	1 (contacts is <i>repeated</i>)
contacts.phoneNumber	2 (phoneNumber is <i>optional</i>)	1 (contacts is <i>repeated</i>)

Per contacts.name

- Max DL = 1: C'è 1 solo campo optional o repeated lungo il percorso (contacts). Poiché name è required, non aggiunge alcun livello di definizione extra oltre al suo contenitore contacts.
- Max RL = 1: C'è 1 solo campo repeated lungo il percorso (contacts).

Per contacts.phoneNumber

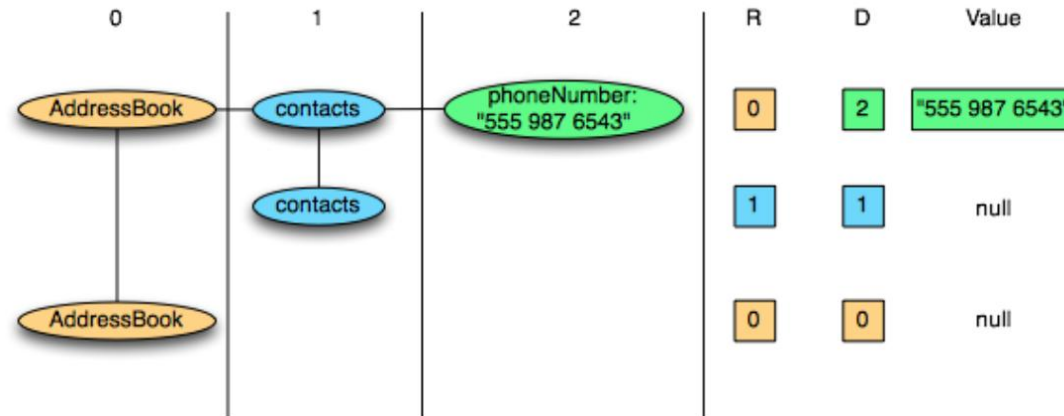
- Max DL = 2: Ci sono 2 campi optional o repeated lungo il percorso: contacts (repeated) e phoneNumber (optional).
- Max RL = 1: C'è 1 solo campo repeated lungo il percorso (contacts). Nota che phoneNumber è optional, non repeated, quindi non incrementa il livello di ripetizione.

```
→ AddressBook {  
  - owner: "Julien Le Dem",  
  [ownerPhoneNumbers: "555 123 4567",  
  ownerPhoneNumbers: "555 666 1337",  
  contacts: {  
    name: "Dmitriy Ryaboy",  
    phoneNumber: "555 987 6543",  
  },  
  contacts: {  
    name: "Chris Aniszczyk"  
  }  
}
```

```
→ AddressBook {  
  - owner: "A. Nonymous"  
}
```

Parquet: AddressBook example

```
AddressBook {  
  contacts: {  
    phoneNumber: "555 987 6543"  
  }  
  contacts: { }  
}  
AddressBook { }
```



Parquet: AddressBook example

To reconstruct the records from the column, we iterate through the column

- **R=0, D=2, Value = "555 987 6543":**
 - R = 0 means a new record.
We recreate the nested records from the root until the definition level (here 2)
 - D = 2 which is the maximum.
The value is defined and is inserted.
- **R=1, D=1:**
 - R = 1 means a new entry in the contacts list at level 1.
 - D = 1 means contacts is defined but not phoneNumber.
We just create an empty contact.
- **R=0, D=0:**
 - R = 0 means a new record.
We create the nested records from the root until the definition level (here 0)
 - D = 0 => contacts is actually null.
We only have an empty AddressBook.

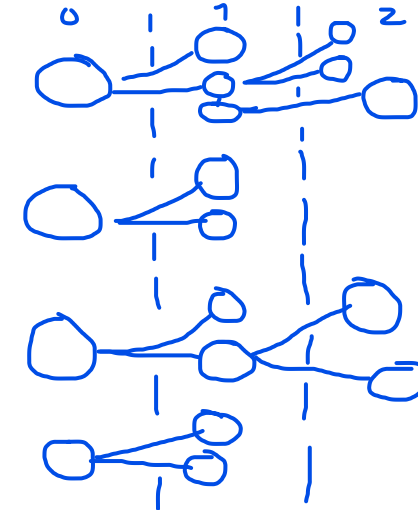
R	D	Value
0	2	"555 987 6543"
1	1	null
0	0	null

```
AddressBook {  
  contacts: {  
    phoneNumber: "555 987 6543"  
  }  
  contacts: { }  
}  
AddressBook { }
```

Parquet: exercise 1

Define the columns (DL, RL, Value)
for **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```



```
[  
  {  
    "name": "Alice",  
    "orders": [  
      { "item": "Book", "quantity": 1 }, item Book: DL=1 ; RL=0 || quantity 1: DL=2 ; RL=0  
      { "item": "Pen", "quantity": null } item Pen: DL=1 ; RL=1 || quantity null: DL=1 ; RL=1  
    ]  
  },  
  {  
    "name": "Bob",  
    "orders": null DL=0 ; RL=0  
  },  
  {  
    "name": "Carol",  
    "orders": [  
      { "item": "Notebook", "quantity": 3 } item Notebook: DL=1 ; RL=0 || quantity 3: DL=2 ; RL=0  
    ]  
  },  
  {  
    "name": "Dan",  
    "orders": [] DL=0 ; RL=0  
  }  
]
```

Parquet: exercise 1 solution

Define the columns (DL, RL, Value)
for **item** and **quantity**

```
message Person {  
  required string name;
```

```
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

DL	RL	item
1	0	Book
1	1	Pen
0	0	Null
1	0	Notebook
0	0	null

```
[  
  {  
    "name": "Alice",  
    "orders": [  
      { "item": "Book", "quantity": 1 },  
      { "item": "Pen", "quantity": null }  
    ]  
  },  
  {  
    "name": "Bob",  
    "orders": null  
  },  
  {  
    "name": "Carol",  
    "orders": [  
      { "item": "Notebook", "quantity": 3 }  
    ]  
  },  
  {  
    "name": "Dan",  
    "orders": []  
  }  
]
```

DL	RL	quantity
2	0	1
1	1	null
0	0	Null
2	0	3
0	0	null

Parquet: exercise 2

Rebuild the messages, starting from the (DL, RL, Value) of **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

DL	RL	item
1	0	A
1	1	B
0	0	null
1	0	C
0	0	null
1	0	D
1	1	E
1	1	F
1	1	G

DL	RL	quantity
1	0	null
2	1	101
0	0	null
2	0	102
0	0	null
2	0	103
1	1	null
2	1	104
1	1	null

Parquet: exercise 2 solution

Rebuild the messages, starting from the (DL, RL, Value) of **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

```
[{ "orders": [  
  { "item": "A", "quantity": null },  
  { "item": "B", "quantity": 101 }  
]},  
{ "orders": null },  
{ "orders": [  
  { "item": "C", "quantity": 102 }  
]},  
{ "orders": null },  
{ "orders": [  
  { "item": "D", "quantity": 103 }  
  { "item": "E", "quantity": null }  
  { "item": "F", "quantity": 104 }  
  { "item": "G", "quantity": null }  
]}  
]
```

DL	RL	item
1	0	A
1	1	B
0	0	null
1	0	C
0	0	null
1	0	D
1	1	E
1	1	F
1	1	G

DL	RL	quantity
1	0	null
2	1	101
0	0	null
2	0	102
0	0	null
2	0	103
1	1	null
2	1	104
1	1	null

Parquet: storing levels, encodings

Definition and repetition levels cause ^{a very little} overhead

- Each primitive type corresponds to three columns Per poter ricostruire le strutture dati annidate senza perdere la gerarchia, Parquet associa ad ogni singola colonna foglia (tipo primitivo) tre informazioni distinte: Il valore effettivo del dato, Il Definition Level (DL) ed Il Repetition Level (RL).
- Their values are bound by the depth of the schema, resulting in **only a few bits used**
- When all fields are *required* in a flat schema, levels can be omitted

Compression takes care of condensing data efficiently

- **Bit Packing**: useful for repetition and definition levels, dictionary keys
- **Run Length Encoding (RLE)**: useful for definition level of sparse columns
- **Dictionary Encoding**: useful for columns with few (<50k) distinct values
- Definition of new encodings is supported

la struttura gerarchica interna di un file Apache Parquet salvato su un filesystem distribuito come HDFS.

Parquet: file format

File: an HDFS file

- Includes the metadata for the file

Row group: a logical horizontal partitioning of the data into rows

Un gruppo di righe (Row Group) è memorizzato per intero dentro un singolo blocco HDFS. (stored in a columnar way inside the block)

- No physical structure is guaranteed for a row group
- Contains a column chunk for each column in the dataset

Column chunk: a chunk of the data for a particular column

lista di valori di una colonna

- These live in a particular row group and are guaranteed to be contiguous in the file

Page: column chunks are divided up into pages

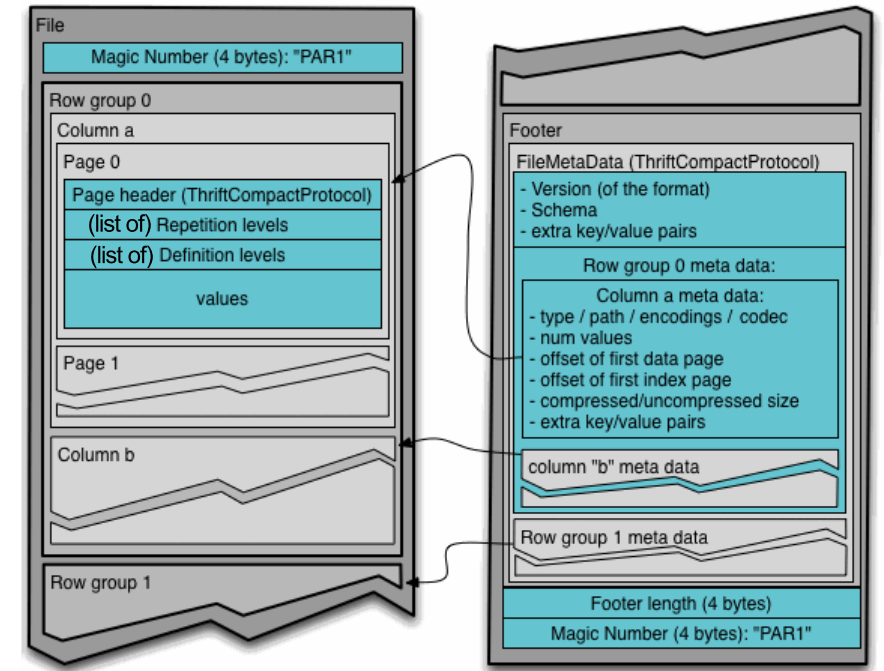
sono suddivisi

- An indivisible unit in terms of compression and encoding

Il formato Parquet combina il partizionamento orizzontale (per righe) con la rappresentazione colonnare (verticale):

- Row Group (Partizionamento orizzontale): Il dataset viene diviso in gruppi di righe. La strategia d'accesso ottimale consiste nel far coincidere ogni Row Group con un singolo blocco HDFS, così che l'intero gruppo di righe risieda fisicamente sullo stesso nodo del cluster.
- Column Chunk (Organizzazione colonnare): All'interno del blocco HDFS, le righe del Row Group vengono salvate per colonna. Per ogni colonna si crea un Column Chunk fisicamente contiguo su disco.
- Page (Sotto-unità di elaborazione): Ciascun Column Chunk è ulteriormente suddiviso in Page, che rappresentano l'unità minima ed indivisibile per la compressione e la codifica dei dati.

Questa struttura gerarchica a tre livelli consente di calibrare (tuning) le dimensioni dei singoli elementi in base al carico di lavoro: Row Group più grandi aumentano le prestazioni delle letture sequenziali ad alto throughput, mentre la dimensione delle Page consente di bilanciare la granularità delle ricerche puntuali con l'overhead della CPU e dei metadati.



Parquet: configuration

Row group size: larger row groups → larger column chunks → larger sequential IO

la dimensione di un Row Group viene fatta coincidere esattamente con la dimensione di un blocco HDFS (ad esempio, impostando entrambi a 1 GB).

- Large row groups (512MB - 1GB) are recommended
 - Since an entire row group might need to be read, we want it to completely fit on one HDFS block
 - Optimized read setup: 1GB row groups, 1GB HDFS block size, 1 HDFS block per HDFS file

each block stores all the data that belongs to a horizontal partition of the table (and then access them in a columnar way)

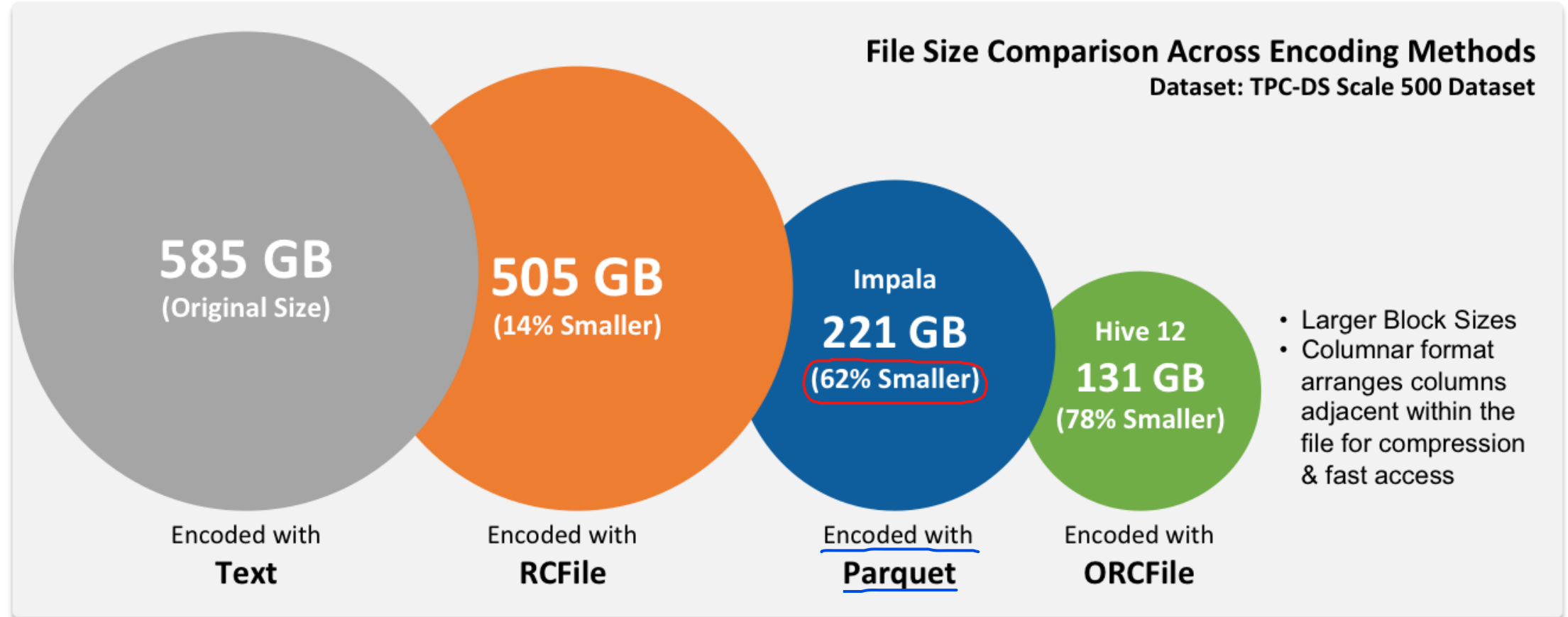
Data page size: data pages should be considered indivisible

- Smaller data pages → more fine grained reading (e.g. single row lookup)
- Larger page sizes → less space overhead (less page headers), less parsing overhead (processing headers)

se le Data Page sono più piccole, la ricerca e la lettura di singole righe specifiche diventano molto più veloci ed efficienti.

se le pagine Parquet sono molto grandi, per accedere a una sola riga (o a un singolo dato) sei costretto a decomprimere e decodificare l'intera pagina che la contiene, elaborando anche tantissimi dati inutili che non ti servono.

Parquet: efficient storage



Data Lake and Lakehouse

Organizing the data in the DFS

Il Data Lake è un archivio centrale basato su un Distributed File System (DFS) in cui vengono raccolti e salvati dati grezzi (raw data) nel loro formato originale, per poi essere elaborati e analizzati solo al momento dell'utilizzo.

The Data Lake

è una metafora (non è un tool)

Place where you put the data in its raw (not processed) original form

The Data Lake is “a central repository system for storage, processing, and analysis of raw data, in which the data is kept in its original format and is processed to be queried only when needed”

In the absence of organization and governance, the Data Lake can easily become a **Swamp**

In assenza di organizzazione, gestione e regole di Data Governance, il lago si trasforma in una palude di dati (Data Swamp). Cosa comporta: Rappresenta un'enorme quantità di dati accumulati in modo disordinato da cui diventa impossibile estrarre informazioni o valore utile per l'azienda.



- Acqua = Dati: I dati grezzi che fluiscono nel sistema.
- Fiumi in entrata (Multiple rivers) = Molteplici fonti di origine: Rappresentano i diversi flussi di dati in arrivo da sorgenti eterogenee (es. email, social media, log, fogli di calcolo).
- Il Lago = Il Data Lake (DFS): Il bacino di archiviazione centrale distribuito dove tutti i dati vengono accumulati così come sono.
- Fiume in uscita (Outgoing river) = Risultati delle analisi: Il flusso di informazioni elaborate e analizzate che esce dal lago per produrre conoscenza e valore di business (insights)

J. Couto, O. T. Borges, D. D. Ruiz, S. Marczak, R. Prikladnicki, *A mapping study about data lakes: An improved definition and possible architectures*, in: Proc. SEKE, Lisbon, Portugal, 2019, pp. 453–578.



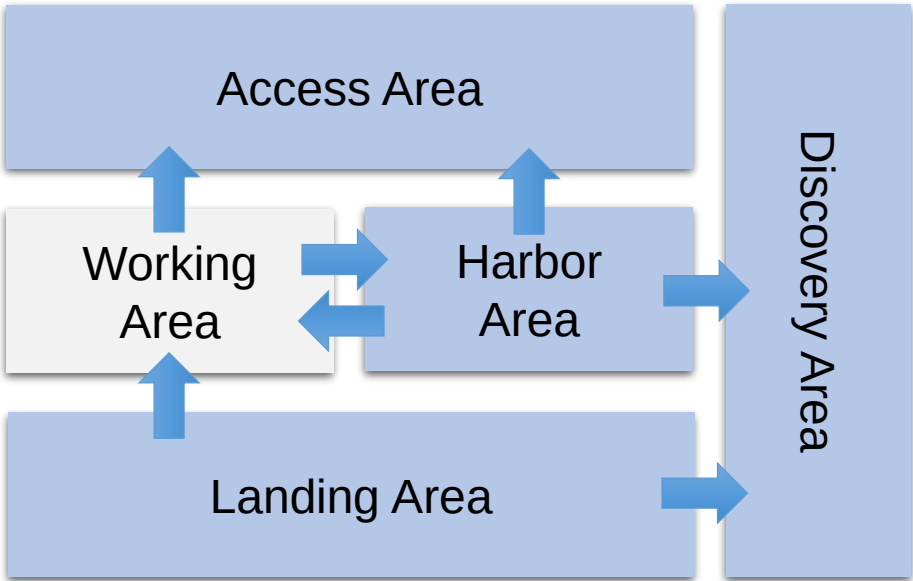
The Data Lake

So, to have Data governance and organization, you need to define which data are raw data and which are processed (don't put them in the same place: you need organization)

To facilitate data governance, the Data Lake can be partitioned into areas, representing the processing state of the data

Contiene unicamente tabelle temporanee, stati intermedi e dati di configurazione.

- **Landing:** where data are stored immediately after ingestion (place where you put raw data get from sources: there can be data that can be obsolete, biased and other properties that can affect the correctness)
- **Working:** it contains temporary tables and configuration data (place where you put temporary data: can be deleted with no issues)
- **Harbor:** it contains data (completely or partially) processed from the landing area
- **Discovery:** it contains data created by data scientists for experimental purposes; it serves as a safe sandbox, where data is only taken from (and not published to) other areas
- **Access:** it contains data ready to be accessed by reporting and visualization tools



- Harbor Area (Area dei Dati Puliti e Affidabili) è la zona in cui i dati grezzi provenienti dalla Landing Area subiscono processi di pulizia, correzione e trasformazione. I dati contenuti diventano affidabili (trusted), poiché vengono corretti gli errori, le anomalie e le incongruenze tipiche dei dati grezzi. Sebbene siano dati puliti e corretti, non sono ancora ideali per l'analisi diretta di business, perché si trovano in un formato dettagliato o riconciliato non modellato per query analitiche veloci.

- Access Area (Area Analitica per Utenti Finali) prende i dati affidabili dall'Harbor Area e li modella, struttura e aggrega appositamente per le esigenze di business. Fornisce dati pronti per essere consultati ed esplorati direttamente dagli utenti finali tramite dashboard, strumenti di reportistica e tool di visualizzazione aziendali.

È un ambiente isolato e sicuro a disposizione di Data Scientist e ricercatori per fare analisi sperimentali, test o addestrare modelli di Machine Learning. Per garantire la governance, vige la regola che i dati possono essere solo letti e presi da altre aree, ma mai pubblicati o re-immessi dalla Discovery Area verso le altre zone aziendali.

Matteo Francia, Enrico Gallinucci, Matteo Golfarelli, Anna Giulia Leoni, Stefano Rizzi, and Nicola Santolini. "Making data platforms smarter with MOSES." *Future Generation Computer Systems* 125 (2021): 299-313.

2

The Data Lake

A common reference today is the **medallion architecture**

- **Bronze**: where ^{raw} data are stored immediately after ingestion; a common practice here is to apply only minimal transformations on the data (e.g., type conversions)
- **Silver**: it contains data processed from the bronze layer; it represents the point at which raw data is filtered, cleaned, and “dressed up” (augmented) by joining across one or many other tables ^{data can be trusted}
- **Gold**: it contains data ready to be accessed by reporting and visualization tools; it is purpose-built to solve explicit intended business goals



Strengtholt, P., 2025. *Building Medallion Architectures: Designing with Delta Lake and Spark*. " O'Reilly Media, Inc."

Data Warehouse

Un Data Warehouse (DWH) è un database centralizzato progettato specificamente per supportare i processi decisionali aziendali. Il passaggio dei dati dai database operativi al DWH risponde a due esigenze fondamentali:

- Isolamento dei carichi di lavoro (Workload Isolation): Le query analitiche richiedono un'elevata potenza computazionale. Separare l'ambiente analitico (DWH) dai database operativi aziendali evita che le analisi di business interferiscano o rallentino le transazioni quotidiane dei sistemi di produzione (OLTP).
- Efficienza su grandi volumi di dati: Nei database operativi i dati sono strutturati per la gestione delle singole transazioni, mentre nel DWH vengono riorganizzati e formattati in strutture ottimizzate per l'analisi aggregata di grandi moli di dati (OLAP)

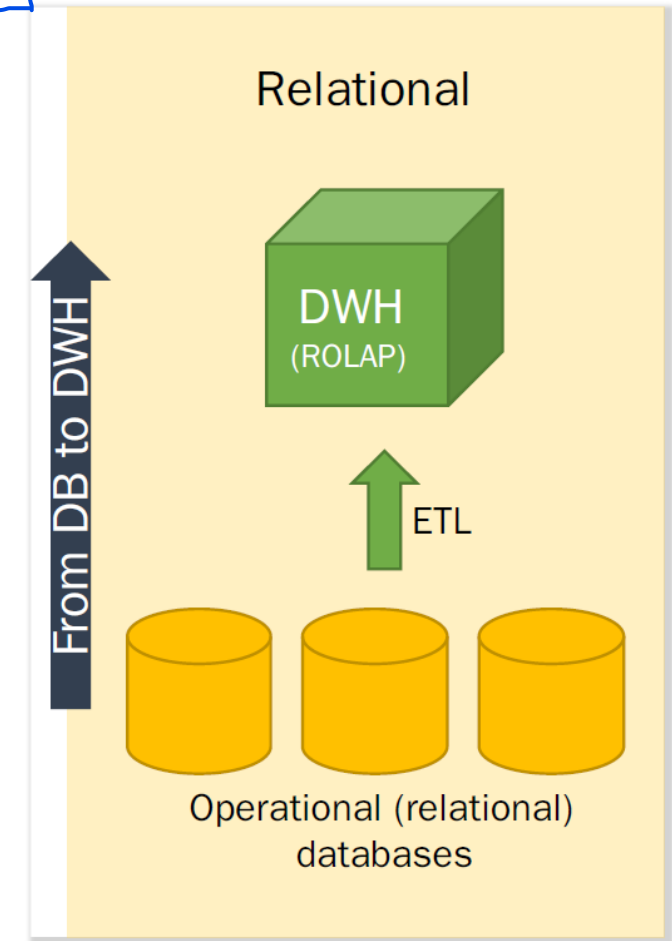
Nelle architetture di Business Intelligence (BI), il trasferimento e la modellazione dei dati avvengono regolarmente tramite pipeline ETL:

- Extract (Estrazione): I dati vengono prelevati dai vari database operativi e sorgenti esterne.
- Transform (Trasformazione): I dati vengono puliti, riconciliati e ristrutturati in un formato ottimizzato per scopi analitici.
- Load (Caricamento): I dati trasformati vengono salvati nel Data Warehouse, pronti per la consultazione senza impattare i sistemi sorgente.

The transformation from raw data to analytics-ready data is similar to what is traditionally done in Business Intelligence and Data Warehousing

A data warehouse is a collection of data that supports decision-making processes

- It is subject-oriented, integrated and consistent, not volatile
- Typically implemented on top of relational DBMSs (ROLAP)



Golfarelli, M. and Rizzi, S., 2009. Data Warehouse design: Modern principles and Methodologies 5 McGraw-Hill. New York.

The Data Lake

Is the medallion architecture just a “revamped” version of the 3-layer data warehouse architecture?

- 1 ▪ Raw area = Bronze level = Source layer
- 2 ▪ Harbor area = Silver level = Reconciled layer
- 3 ▪ Access area = Gold level = Warehouse layer

Some minor differences internally: e.g. the Bronze layer, is a single place where all the data exists; in the source layer, the databases can exist in different organisations and in different physical places

(layer constituted by different databases that can be from different organizations)

(where you put the results of integration of the data that you do from different data sources: layer that can be trusted but not ready yet for the final analysis)

where you put the data that is ready to be accessed from the end users

Perhaps. Ultimately, there is no universal number of areas/levels/layers

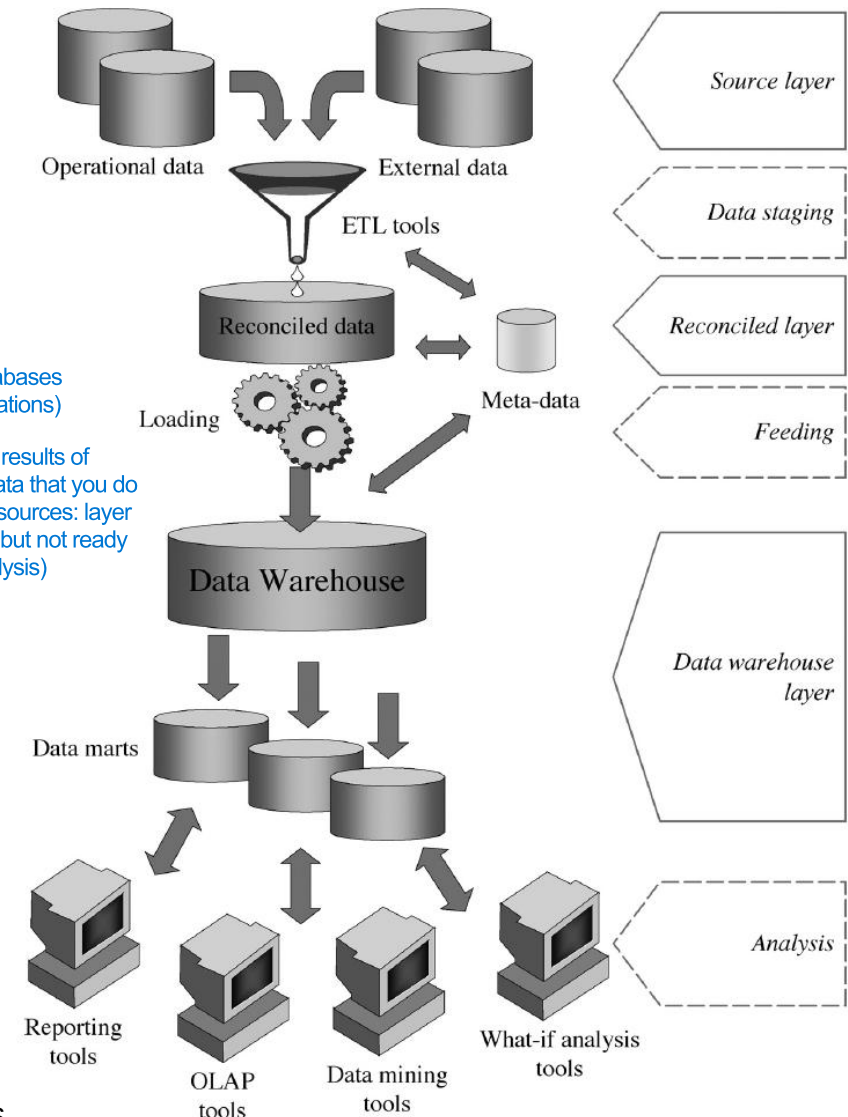
- The basic philosophy is common
- Then, you specialize it based on the complexities within your organization and data pipelines

Il flusso di trasformazione dei dati nell'architettura classica a 3 livelli del Data Warehouse condivide la stessa filosofia di fondo utilizzata per organizzare i Data Lake e la Medallion Architecture:

1° Livello (Dati Grezzi): Raccoglie i dati grezzi direttamente dalle sorgenti senza modifiche strutturali estensive.

2° Livello (Dati Puliti e Riconciliati): I dati vengono filtrati, corretti dagli errori e integrati tra loro per diventare affidabili.

3° Livello (Dati Analitici per il Business): Contiene dati aggregati e modellati appositamente per essere usati da tool di Business Intelligence, dashboard e reportistica finale.



Data Lakehouse

A fusion between a Data Lake and a Data warehouse

Il Data Lakehouse nasce dall'esigenza di accedere e trasformare i dati grezzi dei Big Data per scopi analitici, unendo in un'unica piattaforma i punti di forza del Data Lake e del Data Warehouse. Invece di dover scegliere tra l'ampia scalabilità di un Data Lake o la rigida affidabilità di un Data Warehouse (oppure gestire due sistemi separati collegati da complessi processi ETL), il Lakehouse aggiunge un layer sopra il Data Lake per offrire entrambi i vantaggi.

One way or another, raw big data *will be* accessed and processed (transformed, integrated, enriched) for analytical purposes

- You can use a distributed DBMS for DW (e.g., ClickHouse)
- The recent trend is to make the data lake a *lakehouse*

A data lakehouse is a layer ^{che abilita} ~~that empowers~~ the data lake to become (also) a data warehouse

- ^{unire} ~~The goal is to accommodate both~~ the variety/scalability support of a data lake (which enables data science pipelines) with the reliability/performance of a data warehouse
- Supports OLTP, but mostly meant for OLAP

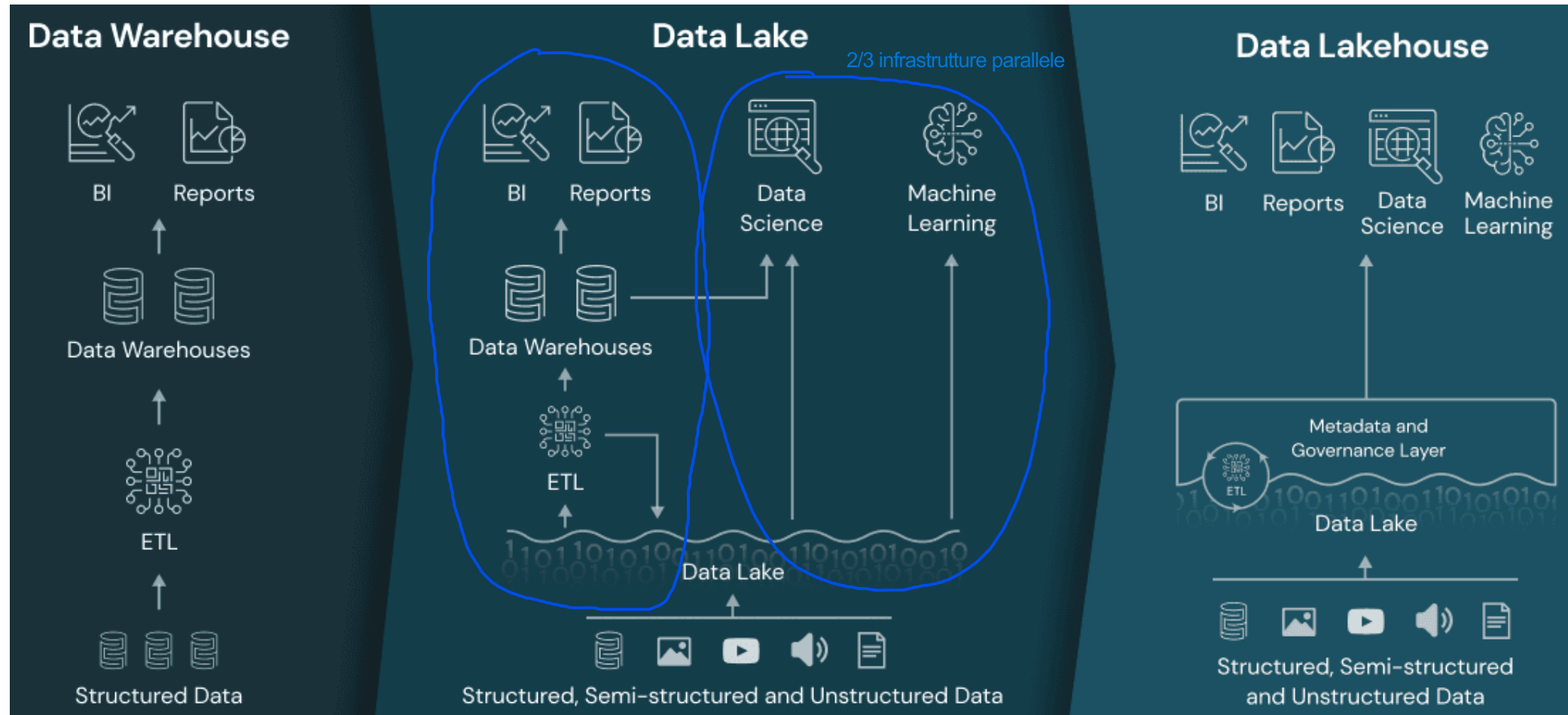
Sebbene supporti operazioni transazionali (OLTP) grazie ai log dei metadati, il Data Lakehouse è nativamente progettato per carichi di lavoro OLAP (Online Analytical Processing), consentendo sia l'esecuzione di query BI e reportistiche tradizionali sia l'analisi avanzata dei dati.

perchè consente di memorizzare nello stesso ambiente distribuito dati strutturati, semi-strutturati e non strutturati, risolvendo le sfide di varietà tipiche dei Big Data.

Introduce la gestione delle transazioni ACID, garantendo la qualità del dato e risolvendo i problemi di inconsistenza (veracity) o il rischio di creare una Data Swamp. Offre elevate prestazioni di calcolo, controllo della sicurezza granulare (a livello di riga e colonna) e rispetto degli schemi (schema enforcement).

Data lakehouse è un layer di meta-dati (dati aggiuntivi oltre a quelli che già abbiamo), che viene messo sopra al Data lake, che ci permette di avere un ambiente in cui possiamo fare in modo più semplice un insieme di attività che va dalla semplice visualizzazione di dashboard ad attività avanzate: unificando la gestione dei dati, fornisce una singola architettura in cui è possibile eseguire in modo semplice sia la reportistica di business (dashboard, BI) sia le analisi avanzate (Data Science, Machine Learning), evitando la complessità di dover costruire e mantenere infrastrutture parallele.

Data Lakehouse (the narrative)



I dati memorizzati sono puliti, completi e sempre coerenti, grazie all'introduzione delle transazioni ACID (Atomicità, Coerenza, Isolamento, Durabilità) fornite dal layer di metadati del Lakehouse.

- Senza ACID (Data Lake tradizionale): Se un job di scrittura fallisce a metà o due processi scrivono contemporaneamente sullo stesso file, si creano dati parziali, corrotti o duplicati, trasformando il sistema in una Data Swamp (bassa qualità).

- Con ACID (Data Lakehouse): Atomicità: Un'operazione di modifica/scrittura avviene completamente o per niente (se fallisce, il sistema annulla tutto senza lasciare dati spazzatura o incompleti). Isolamento: Le letture e le Scritture simultanee non si sovrappongono; chi legge la tabella durante un aggiornamento continua a vedere la versione precedente valida. Risultato: I dati risultano totalmente affidabili e ad alta qualità per il business.

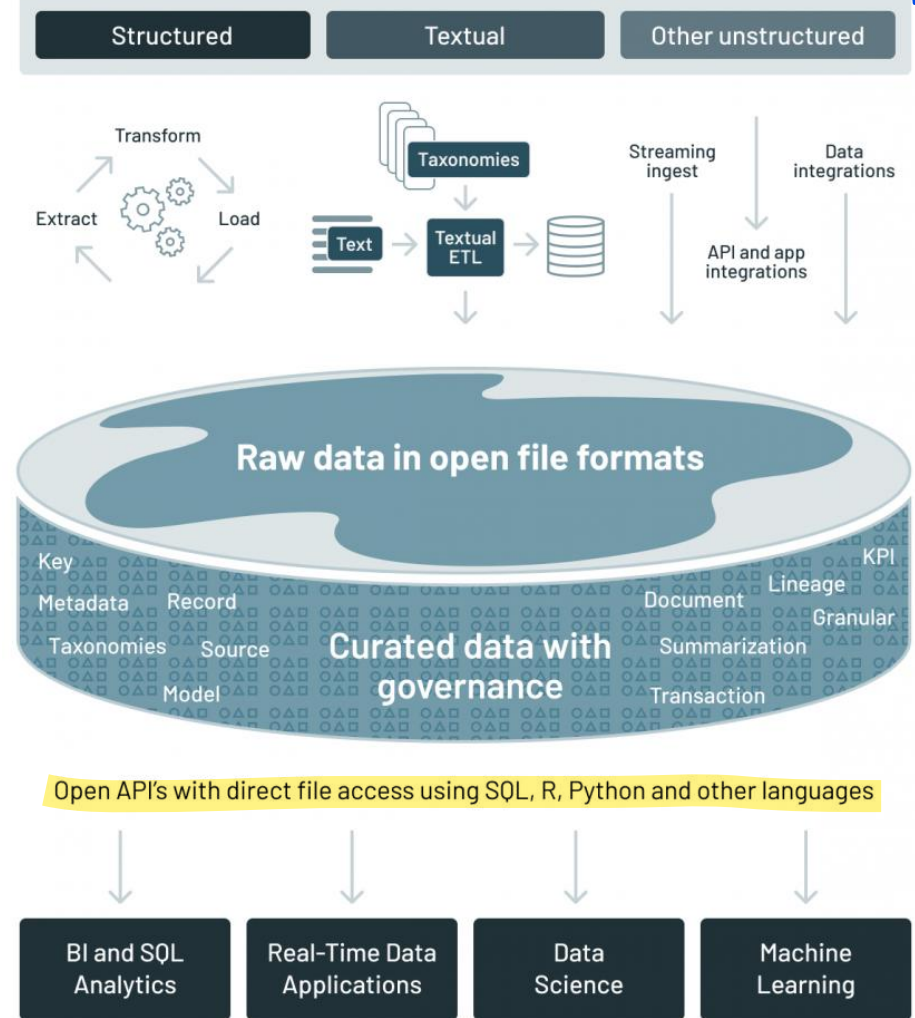
Significa poter applicare regole di sicurezza e permessi di accesso a granolarità fine, controllando la visibilità dei dati fino al dettaglio della singola riga o della singola colonna di una tabella.

Data Lakehouse (the narrative)

Tabella che mostra i vantaggi che Lakehouse prende da entrambi i mondi, eliminandone al contempo i rispettivi limiti.

	Data warehouse	Data lake	Data lakehouse
Data format	Closed, proprietary format	<u>Open format</u> (e.g., <u>Parquet</u>)	Open format (es. <u>Parquet</u>)
Types of data	Structured data, with limited support for semi-structured data	<u>All types</u> : Structured data, semi-structured data, textual data, unstructured (raw) data	All types: Structured data, semi-structured data, textual data, unstructured (raw) data
Data access	SQL-only, no direct access to file	<u>Open APIs</u> for direct access to files with <u>SQL, R, Python</u> and other languages	Open APIs for direct access to files with SQL, R, Python and other languages
Reliability	<u>High quality</u> , reliable data with <u>ACID transactions</u>	Low quality, data swamp	High quality, reliable data with ACID transactions
Governance and security	<u>Fine-grained security and governance</u> for row/columnar level for <u>tables</u>	Poor governance as security needs to be applied to files	Fine-grained security and governance for row/columnar level for tables
Performance	<u>High</u>	Low	High
Scalability	Scaling becomes exponentially more expensive	<u>Scales to hold any amount of data at low cost, regardless of type</u>	Scales to hold any amount of data at low cost, regardless of type
Use case support	Limited to BI, SQL applications and decision support	Limited to machine learning	<u>One data architecture</u> for BI, SQL and <u>machine learning</u>

Parquet è progettato specificamente per dati strutturati (tabelle relazionali) e semi-strutturati (es. file JSON). Non è adatto per dati non strutturati come immagini, file audio, video o documenti PDF grezzi.



<https://databricks.com/blog/2021/05/19/evolution-to-the-data-lakehouse.html>

Data Lakehouse

How it works

- Data is stored with open, columnar formats (mostly Parquet)
 - Data in other formats (e.g., CSV, JSON) must be converted
- A lakehouse needs a table format
 - A metadata layer ontop of the DFS, managing the underlying file format
 - Allows file to be queried as if they were tables (e.g., with SQL)
 - Common table formats: Delta Lake, Apache Iceberg, Apache Hudi

Un file system distribuito (DFS) gestisce solo semplici file e cartelle, non tabelle. Per questo motivo, un Data Lakehouse necessita di un Table Format:

- Layer di metadati sopra il DFS: È uno strato software posizionato al di sopra del file system distribuito che gestisce e organizza i file Parquet sottostanti.
- Astrazione a tabelle: Permette ai motori di calcolo e agli utenti di interrogare i file mediante query SQL come se fossero vere e proprie tabelle di un database.



- The table format provides (Le due funzionalità chiave offerte dal Table Format)
 - A catalog that stores metadata about the files
 - A mechanism to track edits and handle conflicts, enabling ACID transactions

Il Table Format fornisce due componenti essenziali che trasformano il Data Lake in un Lakehouse:

- Catalogo dei metadati (Catalog): Memorizza e aggiorna costantemente le informazioni sui file fisici (schema della tabella, percorsi dei file, partizioni e statistiche).
- Gestione delle modifiche e Transazioni ACID: Fornisce un meccanismo per tracciare gli aggiornamenti/inserimenti e gestire i conflitti di scrittura concorrente. Questo abilita le transazioni ACID (Atomicità, Coerenza, Isolamento, Durabilità), garantendo l'affidabilità del dato anche durante operazioni simultanee di lettura e scrittura.

Data Lakehouse

The organization of the data in the DFS reflects the structure of a DB

- **Based on Apache Hive** Apache Hive organizza il file system ricreando la struttura di un database attraverso una gerarchia di directory

- **/some-dfs-folder/my-database/** È la cartella radice nel DFS che racchiude tutte le tabelle di un determinato ambiente.

Tabella non partizionata

- **./table-A/** Contiene direttamente i file di dati
 - **./fileA1.parquet, ./fileA2.parquet, ...**

Tabella partizionata

- **./table-B/** È suddivisa in sotto-cartelle fisiche per ciascuna partizione
 - **./partition-B1/**
 - **./fileB11.parquet, ./fileB12.parquet, ...**
 - **./partition-B2/**
 - **./fileB11.parquet, ./fileB12.parquet, ...**

Tabella raggruppata per Bucket

- **./table-C/** I dati sono distribuiti in un numero fisso di cartelle o file basati sul calcolo dell'hash
 - **./bucket-C1/**
 - **./fileC11.parquet, ./fileC12.parquet, ...**
 - **./bucket-C2/**
 - **./fileC11.parquet, ./fileC12.parquet, ...**

Based on Apache Hive

Partitioning

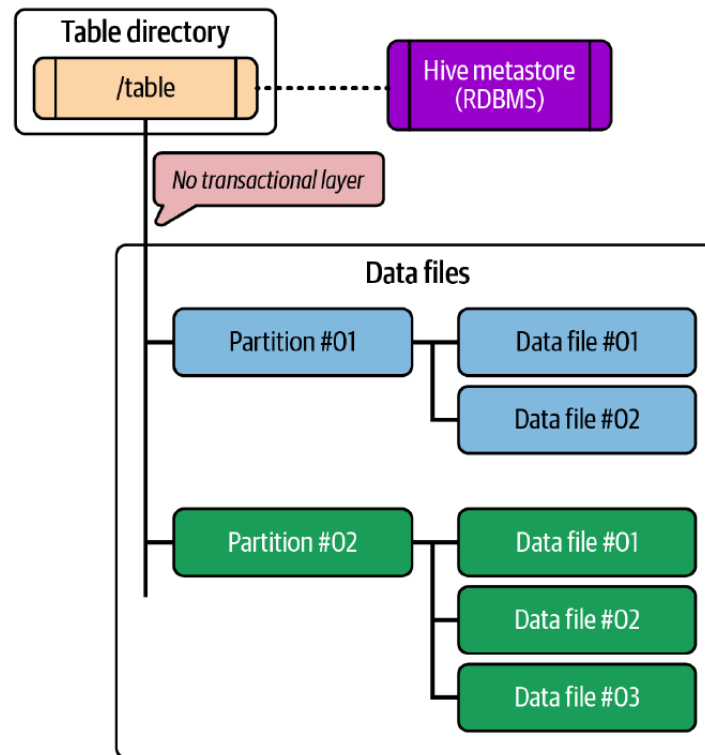
- Stores records based on the value of one (or more) column(s)
- Faster search
- Potential “explosion” of folders
- Ideal for low cardinality columns (e.g., departments)

Bucketing

- Stores records based on the *hash* of a value of one (or more) column(s)
- Faster search (and skipped shuffle)
- The number of buckets is fixed
- Ideal for high cardinality columns (e.g., IDs)

Data Lakehouse

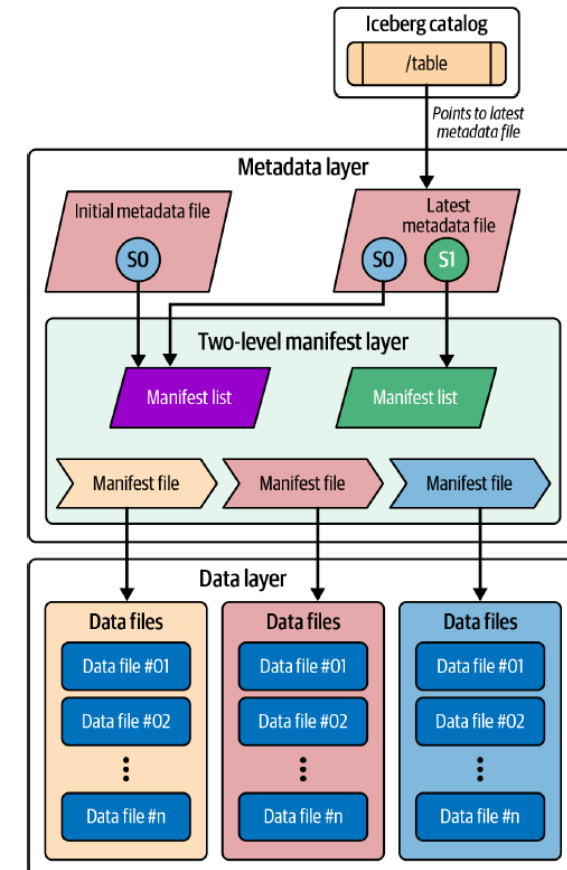
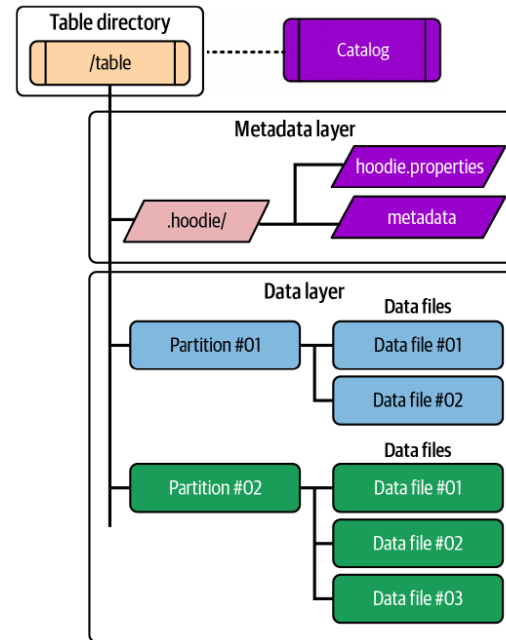
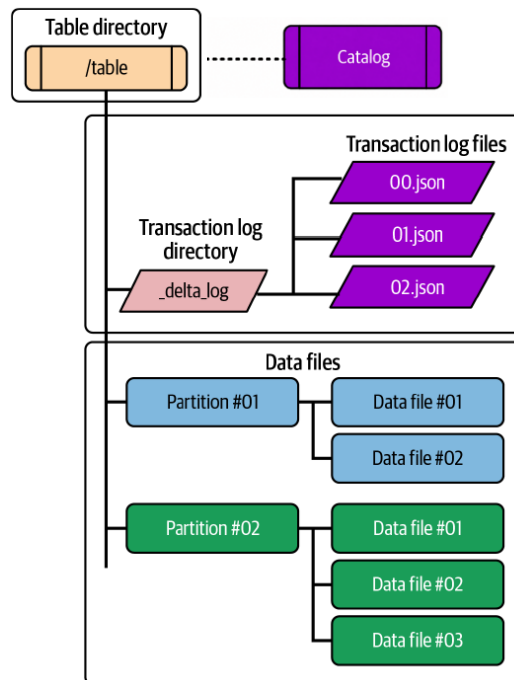
Architecture of Apache Hive



Thalpati, Gaurav Ashok. *Practical Lakehouse Architecture: Designing and Implementing Modern Data Platforms at Scale*. " O'Reilly Media, Inc.", 2024.

Data Lakehouse

Architectures of Delta, Hudi, and Iceberg

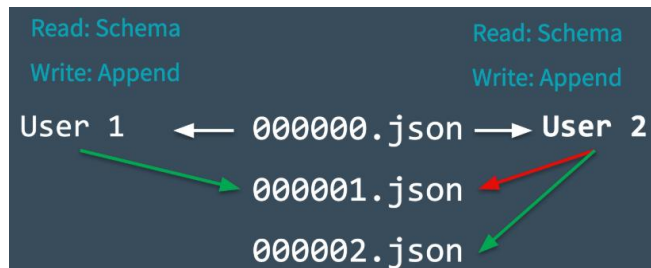


Thalpati, Gaurav Ashok. *Practical Lakehouse Architecture: Designing and Implementing Modern Data Platforms at Scale*. " O'Reilly Media, Inc.", 2024.

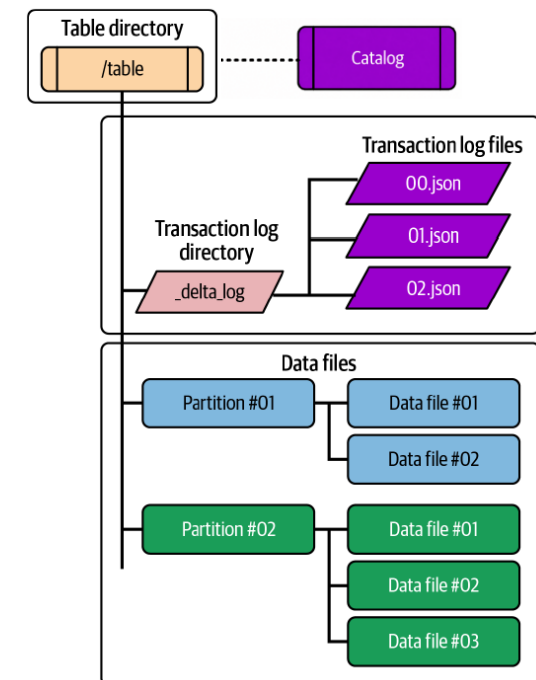
Data Lakehouse

How it works

- In a DFS, files are read-only – so how is data updated?
 - Files are automatically enriched with metadata files, logging all edits
- An *optimistic* concurrency approach is adopted
 - Works on the assumption that conflicts are rare
 - Instead of locking data preemptively, conflicts are checked only at the time of the transaction's commit
 - If a conflict is detected, the system takes corrective actions (e.g., aborting or retrying)



- Works well only with few write transactions per second

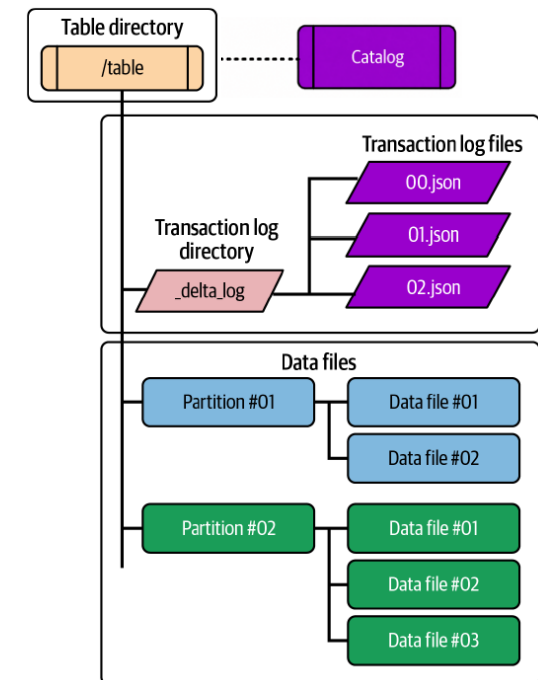
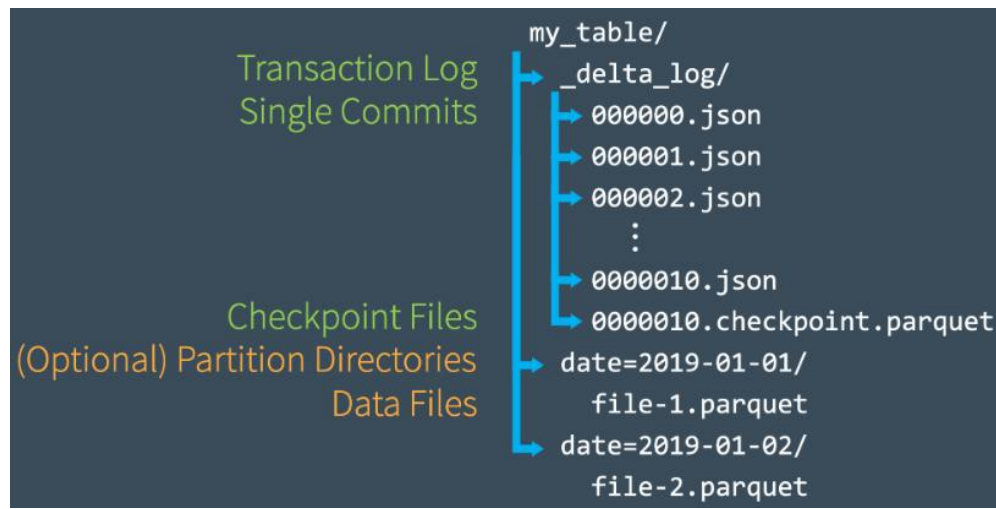


<https://www.databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html>

Data Lakehouse

How it works

- The transaction log scale doesn't scale
 - Checkpointing: materialize the new Parquet file resulting from the application of all edits to the previous file



<https://www.databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html>

Data Lakehouse

How it works

- How is data accessed?
 - The catalog stores metadata about all the files and enables access to files as if they were tables
 - A separate computation engine must be used to access the files
 - The computation engine exploits the lakehouse format's libraries to
 - Update the data with ACID properties
 - Carry out time-travel, i.e., seeing previous versions of the data, up until transaction logs and checkpoints are cleaned (e.g. 30 days)

